



Onboarding guide for AWS Partner Revenue Measurement

AWS Partner Revenue Measurement



AWS Partner Revenue Measurement: Onboarding guide for AWS Partner Revenue Measurement

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

What is Partner Revenue Measurement?	1
Sign up for an AWS account	1
Getting started	2
Partner Revenue Measurement Architecture Patterns	2
Implementation Steps	3
Prerequisites	4
Product Code Retrieval	5
Implementation Methods	6
Methods Overview	6
Choosing the Right Method	7
Decision Tree	8
Special Scenarios	9
Key Considerations	10
AWS Marketplace Metering	11
Method-Specific Requirements	11
Implementation	11
Amazon Machine Image (AMI) products	12
Machine Learning (ML) products	12
Included AWS Services	12
Resource Tagging	14
Method-Specific Requirements	14
Manual Implementation	14
3-Step Implementation Process	14
Tagging via AWS Management Console	15
Tagging via AWS CLI	15
Best Practices	16
Tag Management	16
Automated Tagging	17
AWS CloudFormation	17
AWS Cloud Development Kit (AWS CDK)	18
Terraform	18
AWS Tag Editor	19
AWS CLI	20
API/SDK Bulk Tagging	21

Included AWS Services	22
User Agent String	30
Method-Specific Requirements	30
Manual Implementation	31
Infrastructure as Code Limitations	33
Implementation Samples	34
Python (boto3)	34
Java (AWS SDK v2)	34
.NET (AWS SDK)	35
Node.js (AWS SDK v3)	36
Go (AWS SDK v2)	36
Ruby (AWS SDK v3)	37
Terraform	38
Automated User Agent	39
Shared AWS Config File	40
Environment Variable	40
JVM System Property (Java/Kotlin)	41
Support by AWS SDKs and Tools	41
Validation	41
Included AWS Services	41
Revenue Attribution ID	52
Key Concepts	52
Prerequisites	53
Create and Edit Revenue Attribution IDs	54
Creating a Revenue Attribution ID	54
Adding and editing monthly cost allocation percentages	55
Implementing PRM with a Revenue Attribution ID	55
Resource Tagging	56
User Agent String	57
API Access	57
Partner Revenue Measurement APIs	57
Marketplace Revenue Share APIs	58
Troubleshooting Revenue Attribution IDs	59
Revenue Attribution ID not created successfully	60
Cost allocation percentage entry rejected	60
Unable to update a prior month's cost allocation	60

Offer or opportunity association failing validation	61
Common implementation errors	61
Automation	62
AWS Marketplace Metering	62
Resource Tagging Automation	62
User Agent String Automation	62
Included AWS services	64
Best practices	65
Amazon Bedrock	65
Understanding inference profiles	65
Prerequisites	65
Tagging an application inference profile	66
Listing application inference profiles	68
Amazon EKS	68
Tagging the Kubernetes cluster	69
Tagging nodes within a node group	69
Tagging load balancers	70
Tagging EBS volumes	70
Fargate on EKS	71
Isolating partner solution resources on a shared EKS cluster	71
Demo	72
AWS Samples	73
FAQs	74
Partner FAQ	74
1. General Questions	74
2. Troubleshooting	76
3. Additional FAQs	76
Customer FAQ	76
1. What is Partner Revenue Measurement?	76
2. Why is my Partner asking to implement Partner Revenue Measurement?	76
3. What are the customer benefits of Partner Revenue Measurement?	77
4. What impact does Partner Revenue Measurement have on my AWS environment?	77
5. What specific data does AWS collect?	77
6. If I allow my Partner to implement Partner Revenue Measurement via resource tagging, how does Partner Revenue Measurement impact my tagging strategy?	77
7. How long do resource tags stay in my environment?	78

8. Does Partner Revenue Measurement give my Partner or AWS new access to my account?	78
9. Is my usage shared with my Partner? What information is shared?	78
10. In which AWS regions is Partner Revenue Measurement supported?	78
Troubleshooting	79
AWS Marketplace Metering Issues	79
Resource Tagging Issues	79
Tags not providing revenue attribution	79
Tag conflicts with other partners	80
User Agent String Issues	80
User Agent string not appearing in CloudTrail	80
Common format errors	81
CloudTrail Logs Verification	81
Validation and Testing	82
AWS Partner Team validation	82
Common Implementation Errors	82
Contact Information	83
Document history	84

What is Partner Revenue Measurement?

Partner Revenue Measurement is a set of capabilities that help Partners measure attributed revenue, demonstrate impact, and unlock mutual growth opportunities. It delivers precise, automated measurement of AWS consumption driven by Partner solutions — providing transparent, data-driven recognition of Partner contributions based on actual AWS service consumption across Partner and customer accounts.

After implementing Partner Revenue Measurement, access the Attributed Revenue Dashboard to gain visibility into the revenue impact by Partner product, AWS service, and billing period. See [Attributed Revenue guide](#) for more details.

Sign up for an AWS account

To get started with AWS, you need an AWS account. For information about creating an AWS account, see [Getting started with an AWS account](#) in the *AWS Account Management Reference Guide*.

Getting started with Partner Revenue Measurement

Partner Revenue Measurement measures AWS service consumption driven by partner products. This enables AWS to attribute revenue to partner solutions and provide aggregated consumption data back to partners.

Partner Revenue Measurement supports three implementation methods:

- [AWS Marketplace Metering](#) — Zero-touch revenue attribution for Amazon Machine Image (AMI) and Machine Learning (ML) products listed on AWS Marketplace
- [Resource Tagging](#) — Tag AWS resources with your product code for revenue attribution
- [User Agent String](#) — Include a User Agent string in regular AWS API/CLI calls for revenue attribution

To implement Partner Revenue Measurement, consider the following requirements:

- What AWS services does your product use (Amazon EC2, Amazon S3, Amazon ECS, Amazon RDS)?
- Do you have a SaaS, AMI, ML, or Professional Services product listed on AWS Marketplace?
- Which architecture pattern does your solution follow (Partner account, Customer account, or Hybrid)?

Note

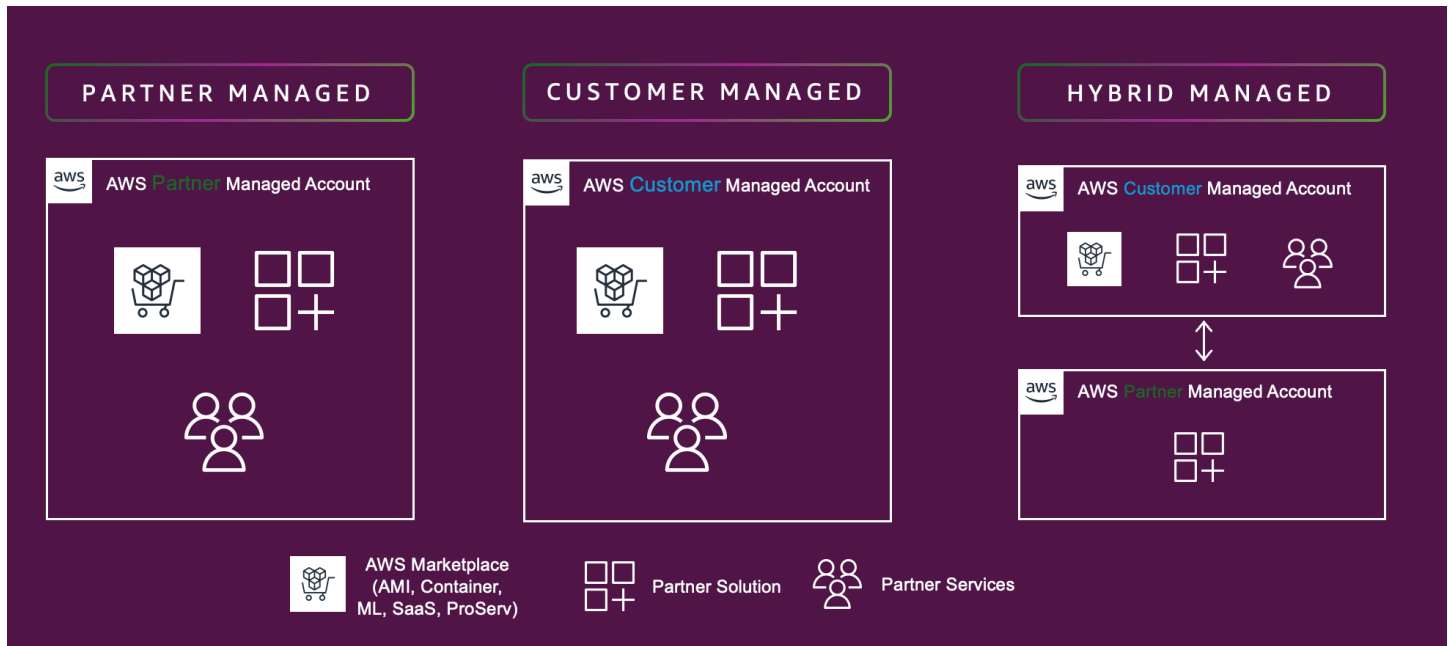
Partner Revenue Measurement requires implementation of one or more methods to enable revenue attribution for [supported AWS services](#).

Partner Revenue Measurement Architecture Patterns

Pattern #1 - Partner Account: All components reside within the partner's AWS account or VPC.

Pattern #2 - Customer Account: All components are deployed in the customer's AWS account and VPC.

Pattern #3 - Hybrid: Components are distributed across both partner and customer AWS accounts and VPCs.



Implementation Steps

Partner Revenue Measurement Implementation Process

1. Step 1: Complete Prerequisites

Review [Prerequisites](#)

2. Step 2: Retrieve Product Code

[Retrieve your product code from AWS Marketplace Management Portal](#)

3. Step 3: Choose Implementation Method

Select one or more [implementation methods](#) based on your product type and architecture

4. Step 4: Complete Method-Specific Requirements and Implement

Review the method-specific requirements for your chosen method, then follow the implementation guide. For a low-effort approach at scale, consider using [automation](#) where available.

Prerequisites

Before implementing Partner Revenue Measurement, you must have:

1. AWS Partner Central and AWS account linking

Before linking accounts, AWS Partners must understand that this linked AWS account will become the primary account for managing all APN activities in Partner Central. This account will:

- Determine Partner Revenue Measurement compliance for APN funding benefits eligibility
- Serve as the primary account for the migration to the new Partner Central experience and all Partner Central users will need to be provisioned access
- Be billed for the annual APN membership fee

When determining which AWS account to link, consider these options:

Option A. Partners with one AWS Marketplace account

Evaluate your existing AWS Marketplace account against the [account selection guidance criteria](#) (Partner Central login required). Link your AWS Marketplace account if you're comfortable with:

- Provisioning Partner Central user access to this account
- Using this account as your primary account for APN engagements
- Being billed the APN membership fee on this account

If you prefer not to designate your AWS Marketplace account as your primary account, create a new AWS account or link an existing non-AWS Marketplace account that meets the above criteria.

Option B. Partners with multiple AWS Marketplace accounts

Evaluate your AWS Marketplace accounts against the [account selection guidance criteria](#) (Partner Central login required).

Note

AWS recommends creating and linking a new AWS account to represent your global business, then connecting all individual AWS Marketplace accounts to your primary account using Subsidiary Account Connections (accessible only after migrating to the new Partner Central experience).

Complete these three steps:

- a. [Create and link](#) an AWS account (which will become your primary account) to your Partner Central account
- b. [Migrate](#) to the new Partner Central experience (Partner Central login required)
- c. Link all AWS Marketplace accounts to your primary AWS account through [Subsidiary Account Connections](#) (Partner Central login required)

2. Product listing on AWS Marketplace

See the [AWS Marketplace Seller Guide](#) for general information. For product-specific listing guide instructions:

Product Type	Listing Guide
SaaS	SaaS Product Listing Guide
Amazon Machine Image (AMI)	AMI Product Listing Guide
Machine Learning (ML)	ML Product Listing Guide
Professional Services	Professional Services Listing Guide

3. Product that uses one or more of the [supported AWS services](#)

4. Ensure that Cost Explorer is enabled

To learn how to enable Cost Explorer, see [Enabling Cost Explorer](#).

Product Code Retrieval

Your AWS Marketplace product code is required for all implementation methods. This code uniquely identifies your product for revenue attribution.

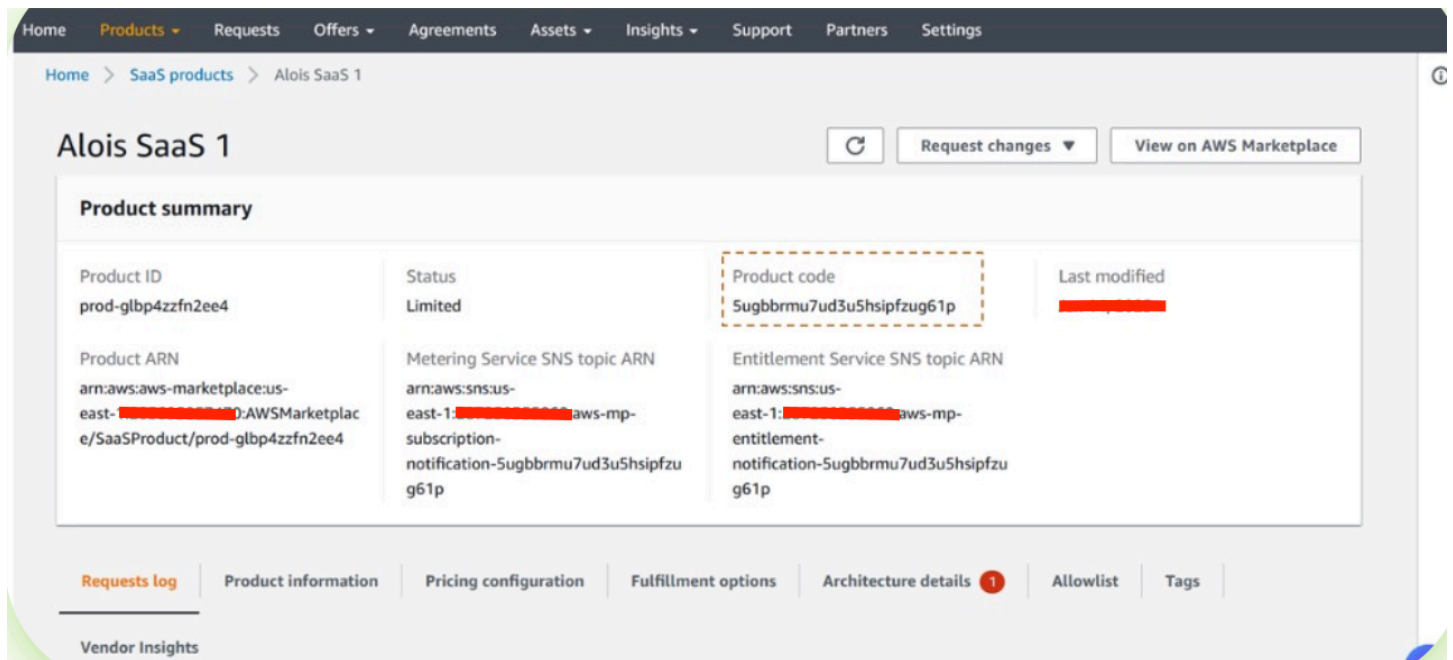
To retrieve your product code

1. Open the [AWS Marketplace Management Portal](#), and then sign in to your seller account

2. Navigate to **Products** page
3. Select your product
4. Find the product code in the **Product Summary** section

Example product code format: 5ugbb1mu7ud3u5hsipfzug61p

The following screenshot shows where to find the product code in the AWS Marketplace Management Portal:



Implementation Methods

Partner Revenue Measurement supports the following implementation methods. Choose the method that best fits your product type and architecture.

Methods Overview

Method	Description	Best For
AWS Marketplace Metering	Zero-touch revenue attribution through AWS Marketplace product metadata	Amazon Machine Image (AMI) and Machine Learning

Method	Description	Best For
		(ML) products listed on AWS Marketplace
Resource Tagging	Tag AWS resources with your product code using the <code>aws-apn-id</code> tag key	SaaS, Professional Services, and any product type where you can tag AWS resources
User Agent String	Include a User Agent string in regular AWS API/CLI calls with your product code	Products with direct regular AWS API/CLI access; AMI/ML products seeking attribution beyond EC2/SageMaker

Note

You can implement multiple methods simultaneously. For example, an AMI product can benefit from AWS Marketplace Metering for EC2 attribution and User Agent String for attribution on additional AWS services used by the product.

Choosing the Right Method

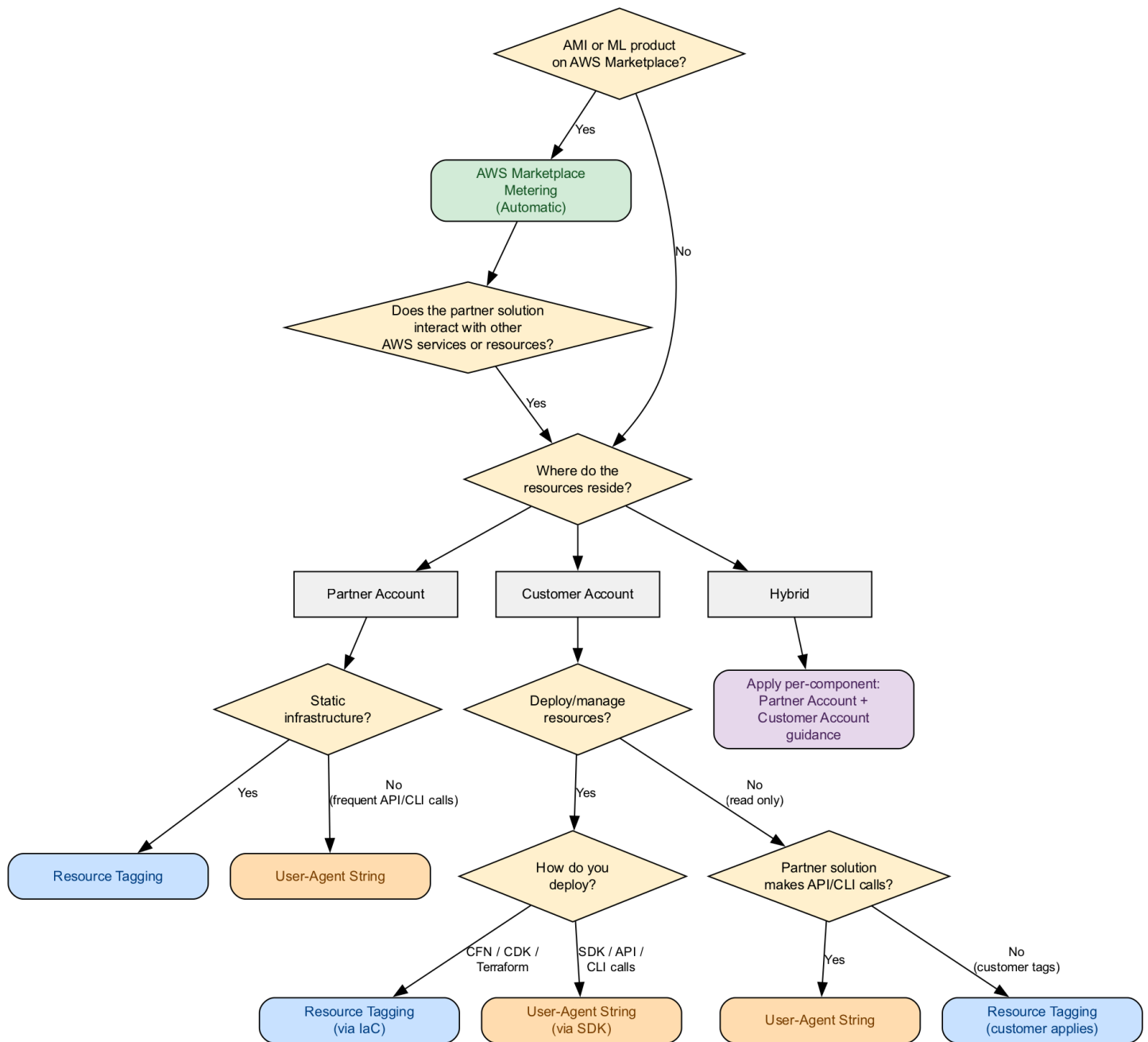
The following table provides guidance on choosing the right method based on your scenario:

Scenario	Use Resource Tagging when...	Use User Agent String when...	Marketplace Metering
Marketplace AMI/ML	—	—	Automatic
Partner Account (SaaS)	Static infrastructure, deploy once and runs long-term	Partner solution makes frequent regular AWS API/CLI calls	—
Customer Account (deploy/manage)	Deploy via IaC (CloudFormation, Terraform), customer allows tags	Partner solution makes ongoing regular AWS API/	—

Scenario	Use Resource Tagging when...	Use User Agent String when...	Marketplace Metering
		CLI calls, or customer has strict tag policies	
Customer Account (read/access only)	Customer willing to tag their resources	Your solution makes read API calls	—
Hybrid (Partner + Customer)	Taggable resources in either account	Partner solution makes regular AWS API/CLI calls in either account	—

Decision Tree

The following decision tree helps you determine the right implementation method:



Special Scenarios

Scenario	Use Resource Tagging when...	Use User Agent String when...	Marketplace Metering
Managed Services Provider (MSP) or	Where write access allows. Provide IaC templates	In management regular AWS API/CLI calls. Modify	—

Scenario	Use Resource Tagging when...	Use User Agent String when...	Marketplace Metering
Systems Integrator (SI/GSI) with limited access	with tags embedded for customer to run	agents or installers to include UA strings	
Multi-partner on same AWS resource	Single <code>aws-apn-id</code> tag limit per resource — may conflict with other partners	Each partner uses own UA string in their respective regular AWS API/CLI calls, no conflicts	—
Backfill existing resources	Bulk tag via Tag Editor or scripted bulk tagging for large-scale backfill. Revenue attribution will only be moving forward from the time tags are applied	Add UA to existing regular AWS API/CLI calls — often the lowest-effort path. Revenue attribution will only be moving forward from the time User Agent strings are implemented	—

Key Considerations

- **Resource Tagging:** Tags are key-value pairs applied to a resource to hold metadata about that resource, in this case a partner identifier. The tag type is a user-defined tag, and can be managed (updated or removed), and counts against the 50-tag-per-resource limit. Only one partner identifier is allowed per resource. Attribution is continuous while the tag is present.
- **User Agent String:** Ephemeral, specific to the regular AWS API/CLI operation carried out on a specific AWS resource, and is read-only. Allows multiple partners to independently use their identifier within their respective regular AWS API/CLI calls. Visible in CloudTrail, enabling customers to govern partner-solution interactions within their AWS resources. Requires at least one regular AWS API/CLI call per resource per month for attribution.
- **Multi-partner:** User Agent String avoids the single `aws-apn-id` tag-per-resource limitation when multiple partners operate on the same AWS resource.
- **Production only:** Partner Revenue Measurement is intended to measure production workloads. Dev/test/staging environments can be used for validating your implementation before rolling out to production.

AWS Marketplace Metering

Partner Revenue Measurement integrates with AWS Marketplace Metering to provide revenue attribution for Amazon Machine Image (AMI) and Machine Learning (ML) products listed on AWS Marketplace.

Method-Specific Requirements

In addition to the general [prerequisites](#), ensure you meet the following requirements for AWS Marketplace Metering:

- Linked AWS Partner Central and AWS account ID. Refer to the [AWS Partner Central and AWS account linking - How to Get Started guide](#) (Requires AWS Partner Central login)
- Product listing on AWS Marketplace. See the [AWS Marketplace Seller Guide](#) for general information. For product-specific listing guide instructions:

Product Type	Listing Guide
Amazon Machine Image (AMI)	AMI Product Listing Guide
Machine Learning (ML)	ML Product Listing Guide

- Product is one of the following AWS Marketplace product types only: Amazon Machine Image (AMI) or Machine Learning (ML)
- Product needs to be purchased and used by buyers on AWS Marketplace

Implementation

Partner Revenue Measurement integrates with AWS Marketplace Metering to measure service usage and attribute the consumption to your product, without requiring any additional inputs such as tagging resources or implementing user agents. This is a zero-touch experience that leverages existing product metadata for revenue attribution.

To avail attribution via this method, you need to list your AMI or ML product on AWS Marketplace and ensure customers purchase and use it via AWS Marketplace.

Amazon Machine Image (AMI) products

Revenue attribution for AMI products works as follows:

1. When you list an AMI product on AWS Marketplace, a unique product code is assigned to it.
2. This product code is automatically attached as instance metadata to every Amazon EC2 instance launched from your AMI.
3. When a buyer purchases and starts using your AMI product, Partner Revenue Measurement uses the product code to measure Amazon EC2 usage and attribute the consumption to your product.

Refer to [Understanding AMI-based products and AMI product codes](#) for more details.

Machine Learning (ML) products

When a buyer purchases, deploys, and starts consuming an ML product, AWS Marketplace can measure the usage of the product to bill the customer. Partner Revenue Measurement integrates with AWS Marketplace Metering to measure the customers' usage of Amazon SageMaker AI resources created by your product and automatically attributes the consumption to your product.

Included AWS Services

Partner Revenue Measurement automatically measures service consumption for Amazon Machine Image (AMI) and Machine Learning (ML) products listed on AWS Marketplace. The following services are supported:

AWS Marketplace Metering Included Services

Service Name	Product Service Code	Notes
Amazon EC2	AmazonEC2	Amazon Machine Image (AMI) products
Amazon SageMaker AI	AmazonSageMaker	Machine Learning (ML) products

Note

Partner Revenue Measurement intends to support all AWS services. We recommend that you instrument PRM on all AWS services and resources that your partner solution interacts with to avoid on-going operational changes as service coverage expands. At this time, revenue attribution data is surfaced for the services listed above. Any partial spend captured on services not listed above is aggregated as "Other".

Resource Tagging

Partner product resource tagging enables AWS Partners to demonstrate how their products drive service consumption and AWS revenue by tagging resources with partner product identifiers.

This self-service capability allows partners to tag resources in their own account or customer's account to measure AWS revenue driven by the partner's product.

Method-Specific Requirements

In addition to the general [prerequisites](#), ensure you meet the following requirements for resource tagging:

- Ability to tag AWS resources in your own account or in customer's account
- Product that uses one or more of the [supported AWS services for resource tagging](#)

Manual Implementation

Note

Partner Revenue Measurement is intended to measure production workloads. Dev/test/staging environments can be used for validating your implementation before rolling out to production.

3-Step Implementation Process

1. [Get your product code from AWS Marketplace Management Portal](#)
2. Add the tag to your resources:
 - Tag Key: `aws-apn-id`
 - Tag Value: `pc:<product-code>`
 - Example: `pc:5ugbb1mu7ud3u5hsipfzug61p`
3. Apply via automated or manual methods listed below

 Note

Ensure the `aws-apn-id` tag fits within the [50-tag-per-resource limit](#) and does not conflict with existing customer tag policies.

Tagging via AWS Management Console

You can manually tag your resources using the AWS Management Console.

To get started

1. Go to your AWS Management Console.
2. Go to the resources you want to tag. Example: Amazon RDS.
3. Choose **Add tags**.
4. Enter `aws-apn-id` as the **Tag key**.
5. Enter `pc:5ugbb1mu7ud3u5hsipfzug61p` as the **Tag value** (replace `5ugbb1mu7ud3u5hsipfzug61p` with your product code).
6. Choose **Save**.

Repeat the steps above for all associated resources such as Snapshots. For more information about tagging resources, see the [Tag your Amazon EC2 resources](#) in the Amazon Elastic Compute Cloud user guide for Linux instances.

 Important

If the resource is managed by infrastructure as code (CloudFormation, Terraform, CDK), adding tags via the console causes drift detection on the next IaC run. For IaC-managed resources, always apply tags through the IaC tool instead.

Tagging via AWS CLI

You can tag specific resources using the AWS CLI.

Note

Replace 5ugbb1rmu7ud3u5hsipfzug61p with your product code in the following example.

```
aws ec2 create-tags --resources i-1234567890abcdef0 \
--tags Key=aws-apn-id,Value=pc:5ugbb1rmu7ud3u5hsipfzug61p
```

Important

If the resource is managed by infrastructure as code (CloudFormation, Terraform, CDK), adding tags via CLI causes drift detection on the next IaC run. For IaC-managed resources, always apply tags through the IaC tool instead.

Best Practices

- Always use lowercase 'aws-apn-id' key
- Ensure 'pc:' prefix in value
- Validate product code format
- Document all tagged resources
- Monitor tag compliance and revenue attribution

Note

Once a resource is tagged with a partner's product code, AWS continues to attribute revenue to the product until the tag is removed or the resource is shut down.

Tag Management

Tag Conflicts: Since an AWS resource can only have one tag with the `aws-apn-id` key, only one partner identifier is allowed per resource. For multi-partner scenarios, consider using the [User Agent String](#) method instead. If you must use resource tagging, coordinate with the other partner and the customer to determine tag ownership before making changes.

Tag Removal: Any user with account access can remove tags. Both customers and partners (with account access) can remove tags.

Automated Tagging

We encourage you to automate your tagging resources as much as possible. You can automate your tagging by adding a tag line to your AWS CloudFormation template, AWS Cloud Development Kit (AWS CDK), Terraform, or by using the AWS Tag Editor.

Warning

Only tag resources that are directly used or influenced by your partner solution.

AWS CloudFormation

Stack-level propagation (recommended): Tags applied at the stack level propagate automatically to all resources in the stack that support tagging. This is the simplest approach when your entire stack belongs to your partner solution.

Note

Replace 5ugbbrmu7ud3u5hsipfzug61p with your product code in the following examples.

```
aws cloudformation create-stack --stack-name my-stack \  
  --template-body file:///template.yaml \  
  --tags Key=aws-apn-id,Value=pc:5ugbbrmu7ud3u5hsipfzug61p
```

Note

Most common resource types (EC2, S3, RDS, Lambda, ECS, etc.) support stack-level tag propagation. However, some resources do not — for example, EBS volumes created from block device mappings. Use per-resource tagging as a fallback for those specific resources.

Per-resource tagging: Add the tag directly to individual resource definitions when you need to tag only specific resources within a stack, or when different resources need different attribution.

```
Resources:
  MyResource:
    Type: 'AWS::EC2::Instance'
    Properties:
      Tags:
        - Key: "aws-apn-id"
          Value: "pc:5ugbb1mu7ud3u5hsipfzug61p"
```

For information about which resources you can tag with CloudFormation, see the [AWS resources and property types reference](#) in the AWS CloudFormation user guide.

AWS Cloud Development Kit (AWS CDK)

A tag in AWS CDK is applied to a given construct and all of its taggable children. For more information, see [Tagging](#) in the AWS CDK v2 developer guide.

Stack-level propagation (recommended): Tag the stack to propagate to all resources.

Note

Replace 5ugbb1mu7ud3u5hsipfzug61p with your product code in the following examples.

```
Tags.of(stack).add('aws-apn-id', 'pc:5ugbb1mu7ud3u5hsipfzug61p');
```

Per-resource tagging:

```
{
  "Key" : "aws-apn-id",
  "Value" : "pc:5ugbb1mu7ud3u5hsipfzug61p"
}
```

Terraform

Provider-level default tags (recommended): The `default_tags` block applies the tag to every resource Terraform creates or manages through that provider.

Note

Replace 5ugbb1rmu7ud3u5hsipfzug61p with your product code in the following examples.

```
provider "aws" {  
  default_tags {  
    tags = {  
      aws-apn-id = "pc:5ugbb1rmu7ud3u5hsipfzug61p"  
    }  
  }  
}
```

Note

Some Terraform resource types do not support `default_tags` (e.g., `aws_autoscaling_group`). If you also define the same tag key in a resource's `tags` block, Terraform raises a conflict. Use either `default_tags` or per-resource tags for the `aws-apn-id` tag, not both on the same resource.

Per-resource tagging: Add the tag to individual resource blocks when you need to tag only specific resources.

```
resource "aws_instance" "example" {  
  tags = {  
    aws-apn-id = "pc:5ugbb1rmu7ud3u5hsipfzug61p"  
  }  
}
```

AWS Tag Editor

You can use the AWS Resource Groups and Tag Editor to tag resources in bulk using the console.

 Important

For resources provisioned by infrastructure as code templates (CloudFormation, CDK, Terraform, etc.), it is recommended to update the templates instead of using Tag Editor.

 Note

The AWS Tag Editor only works for resources running in the account.

To get started

1. Open the AWS Management Console.
2. Go to the Resource Groups & Tag Editor, Tagging, Tag Editor page.
3. Specify the Region(s) your resources are located. Example: us-east-1.
4. Choose the type of resources you want to bulk tag. Example: EC2, Lambda, and S3.
5. Choose **Search resources** to view the resources that meet the conditions you have selected.
6. Select all or a few of the listed resources you want to tag.
7. Choose **Manage tags of selected resources**.
8. Enter `aws-apn-id` in the **Tag Key** field.
9. Enter `pc:5ugbb1mu7ud3u5hsipfzug61p` in the **Tag Value** field (replace `5ugbb1mu7ud3u5hsipfzug61p` with your product code).
10. Choose **Review**.
11. Choose **Apply tag changes**.

AWS CLI

You can use the AWS CLI to tag resources in bulk using the command line.

 Important

For resources provisioned by infrastructure as code templates (CloudFormation, CDK, Terraform, etc.), it is recommended to update the templates instead of using CLI commands.

Note

Replace 5ugbbrmu7ud3u5hsipfzug61p with your product code in the following example.

```
aws resourcegroupstaggingapi tag-resources \  
  --resource-arn-list arn:aws:ec2:region:account-id:instance/i-1234567890abcdef0 \  
  --tags aws-apn-id=pc:5ugbbrmu7ud3u5hsipfzug61p
```

API/SDK Bulk Tagging

Use the Resource Groups Tagging API to tag multiple resources across different service types in a single call. This is the most efficient approach for large-scale tagging.

Note

Replace 5ugbbrmu7ud3u5hsipfzug61p with your product code in the following example.

```
import boto3  
  
tagging = boto3.client('resourcegroupstaggingapi')  
tagging.tag_resources(  
    ResourceARNList=[  
        'arn:aws:ec2:us-east-1:123456789012:instance/i-1234567890abcdef0',  
        'arn:aws:s3:::my-bucket'  
    ],  
    Tags={'aws-apn-id': 'pc:5ugbbrmu7ud3u5hsipfzug61p'}  
)
```

⚠ Important

For resources managed by infrastructure as code (CloudFormation, CDK, Terraform), always apply tags through the IaC tool instead. SDK/CLI tagging of IaC-managed resources causes drift detection on the next IaC run.

Included AWS Services

The following AWS services are supported for resource tagging implementation. Resources must be tagged with key `aws-apn-id` and value `pc:product-code` format.

For a downloadable version of this list, download the [Resource Tagging Included Services \(CSV\)](#).

Resource Tagging Included Services

Service Name	Product Service Code	Notes
Amazon API Gateway	AmazonApiGateway	None
Amazon AppStream	AmazonAppStream	Excludes User Fees
AWS AppSync	AWSAppSync	None
Amazon Athena	AmazonAthena	None
Aurora DSQL	AuroraDSQL	None
AWS Backup	AWSBackup	None
Amazon Bedrock	AmazonBedrock	Includes only Amazon models and Open Source models available in Amazon Bedrock
Amazon Bedrock AgentCore	AmazonBedrockAgentCore	Includes AgentCore RunTime, Browser Tool, Code Interpreter, Gateway, Identity, and Memory
AWS Certificate Manager	AWSCertificateManager	Includes AWS Certificate Manager Private CA
Amazon Cloud Directory	AmazonCloudDirectory	None
AWS Cloud WAN	AWSCloudWAN	Excludes Core Network Edge Hours and data transfer

Service Name	Product Service Code	Notes
Amazon CloudFront	AmazonCloudFront	Excludes data transfer costs and Lambda@Edge
AWS CloudHSM	CloudHSM	None
Amazon CloudWatch	AmazonCloudWatch	Logs only
AWS CodeBuild	CodeBuild	None
AWS CodePipeline	AWSCodePipeline	None
AWS CodeStar	AWSCodeStar	None
Amazon Cognito	AmazonCognito	Excludes Amazon Cognito add-ons
Amazon Comprehend	comprehend	None
Amazon Connect	AmazonConnect	Includes Full Connect Unlimited AI and A La Carte; excludes Cases, Entity Resolution, Legacy Pinpoint Engagement, Meetings SDK, ContactLens, Chat, Email, Lex, Q in Connect, Tasks, Voice, Customer Profiles, Outbound Campaigns Processing, Telephony
AWS Data Pipeline	datapipeline	None
AWS Database Migration Service	AWSDatabaseMigrationSvc	None
AWS DataSync	AWSDataSync	None
AWS Deadline Cloud	AWSDeadlineCloud	Excludes AWS data transfer charges and Bring-Your-Own-License third-party creative tool software license costs

Service Name	Product Service Code	Notes
AWS Direct Connect	AWSDirectConnect	Excludes AWS Local Zones
AWS Directory Service	AWSDirectoryService	None
Amazon DocumentDB (with MongoDB compatibility)	AmazonDocDB	None
Amazon DynamoDB	AmazonDynamoDB	None
Amazon DynamoDB Accelerator (DAX)	AmazonDAX	None
Amazon EC2	AmazonEC2	Includes Amazon EBS, EBS Snapshots, EC2 Mac, AWS Local Zones deployment, Savings Plan for ML instance types; excludes Capacity Block for ML
Amazon ECR	AmazonECR	None
Amazon ECS	AmazonECS	Includes Fargate and AWS App Mesh
Amazon EKS	AmazonEKS	Excludes Fargate; includes AWS App Mesh
Amazon Elastic VMware Service (Amazon EVS)	AmazonEC2	Only includes underlying Amazon EC2 use; excludes VCF licenses, VPC Route Server Endpoints, EVS control plane
AWS Elastic Beanstalk	AWSElasticBeanstalk	None
AWS Elastic Disaster Recovery (DRS)	AWSElasticDisaster Recovery	None
Amazon Elastic File System	AmazonEFS	None
Amazon ElastiCache	AmazonElastiCache	None

Service Name	Product Service Code	Notes
AWS Elemental MediaConvert	AWSElementalMediaConvert	None
AWS Elemental MediaLive	AWSElementalMediaLive	None
AWS Elemental MediaPackage	AWSElementalMediaPackage	None
Amazon EMR	ElasticMapReduce	Includes AWS Local Zones deployment
AWS End User Messaging	AmazonPinpoint	Excludes Carrier Fees, Carrier Fee Count, Number Information (Carrier Lookup), Numbers Validated, Phone numbers, SMS Inbound Price
Amazon FinSpace	AmazonFinSpace	Excludes any kdb Insights software license amount
Amazon FSx	AmazonFSx	None
Amazon GameLift	AmazonGameLift	Excludes GameLift Anywhere, FleetIQ, and FlexMatch when using either EC2 for GameLift or On-premises for GameLift
AWS Glue	AWSGlue	None
AWS HealthImaging	AmazonMedicalImaging	None
AWS HealthLake	AmazonHealthLake	Excludes FHIR data export and transformation
AWS IoT Core	AWSIoT	Excludes Registry operations

Service Name	Product Service Code	Notes
AWS IoT SiteWise	AWSIoTSiteWise	Excludes Alarm, Query, Data Storage - Warm Storage, Edge - Data collection pack, Assistant - Monthly Enablement Fee, Assistant - API bundle price, Messaging - Bulk Import, Data Export - Bulk Import
Amazon Kendra	AmazonKendra	None
AWS Key Management Service	awskms	Excludes cross-account request costs
Amazon Keyspaces (for Apache Cassandra)	AmazonMCS	None
Amazon Kinesis Data Analytics	AmazonKinesisAnalytics	None
Amazon Kinesis Data Firehose	AmazonKinesisFirehose	None
Amazon Kinesis Data Streams	AmazonKinesis	None
Amazon Kinesis Video Streams	AmazonKinesisVideo	None
AWS Lambda	AWSLambda	None
Elastic Load Balancing	AWSELB	None
AWS Mainframe Modernization	AWSM2	Excludes 'M2 Custom' and Blu Age Transformation Center costs
Amazon MemoryDB for Redis	AmazonMemoryDB	Excludes Snapshot Storage
Amazon MQ	AmazonMQ	None

Service Name	Product Service Code	Notes
Amazon MSK	AmazonMSK	None
Amazon Neptune	AmazonNeptune	None
AWS Network Firewall	AWSNetworkFirewall	None
Amazon Omics	AmazonOmics	None
Amazon OpenSearch Service	AmazonES	Includes Amazon Elasticsearch Service; excludes OpenSearch Serverless and OpenSearch Ingestion
AWS Payment Cryptography	PaymentCryptography	Excludes following APIs under Requests usage: ListAliases, ListKeys, GetResourcePolicy, GetParametersForImport, GetParametersForExport, DeleteResourcePolicy, DeleteAlias, CreateKey, CreateAlias
Amazon QuickSight	AmazonQuickSight	Excludes Region fee for Q, SPICE, unused charges on subscription packs, Pro user, Pro author, and administrator
Amazon Redshift	AmazonRedshift	Amazon Redshift Provisioned and Amazon Redshift Serverless
Amazon Relational Database Service (RDS)	AmazonRDS	Includes all RDS engines, Amazon RDS Custom, AWS Local Zones deployment; excludes Db2 licensing fees
AWS Resilience Hub	AWSResilienceHub	None
Amazon Route 53	AmazonRoute53	Excludes Amazon Route 53 Resolver, Traffic Flow, and CIDR block storage

Service Name	Product Service Code	Notes
Amazon S3	AmazonS3	Includes storage cost only and all storage tiers; excludes Requests
Amazon S3 Glacier	AmazonGlacier	Excludes Glacier Deep Archive
Amazon SageMaker AI	AmazonSageMaker	Excludes Amazon SageMaker AI training plans for training jobs or HyperPod clusters
AWS Secrets Manager	AWSSecretsManager	None
AWS Security Hub	AWSSecurityHub	None
Amazon Simple Notification Service (SNS)	AmazonSNS	None
Amazon Simple Queue Service (SQS)	AWSQueueService	None
AWS Step Functions	AmazonStates	None
AWS Storage Gateway	AWSSStorageGateway	None
AWS Systems Manager	AWSSystemsManager	OpsCenter only
Amazon Timestream	AmazonTimestream	None
AWS Transfer Family	AWSTransfer	Excludes AWSDataTransfer
AWS Transit Gateway	AmazonVPC	Includes Transit Gateway VPN, VPC, Peering, DirectConnect, DX; excludes data transfer costs
Amazon VPC Lattice	AmazonVPC	None

Service Name	Product Service Code	Notes
Amazon WorkSpaces	AmazonWorkSpaces	Includes WorkSpaces Core; excludes third-party VDI management software license and third-party OS or software license on virtual desktop
Amazon WorkSpaces Core Managed Instances	AmazonWorkSpacesInstances	Excludes hourly metering, third-party VDI management software license, third-party OS or software license on virtual desktop

Note

Revenue attribution continues until the tag is removed or the resource is terminated.

Note

Partner Revenue Measurement intends to support all AWS services. We recommend that you instrument PRM on all AWS services and resources that your partner solution interacts with to avoid on-going operational changes as service coverage expands. At this time, revenue attribution data is surfaced for the services listed above. Any partial spend captured on services not listed above is aggregated as "Other".

User Agent String

This guide provides step-by-step instructions for AWS Partners to implement User Agent strings for Partner Revenue Measurement revenue attribution using AWS APIs and SDKs.

Note

If you have an AMI product listed on AWS Marketplace that uses services beyond Amazon EC2, or an ML product that uses services beyond Amazon SageMaker AI, AWS recommends integrating a User Agent string to receive additional revenue attribution for your product.

Note

Required Format: APN_1.1/pc_<YOUR-PRODUCT-CODE>\$

The User Agent string format must be included in all regular AWS API/CLI calls made by your solution. Where <YOUR-PRODUCT-CODE> is your alphanumeric product code from AWS Marketplace and \$ is the required end delimiter.

Note

For revenue attribution, your product must conduct at least one API operation on an AWS Resource Name (ARN) per month. If no API operations are performed on a resource in a given month, that resource does not contribute to revenue attribution for that month.

Method-Specific Requirements

In addition to the general [prerequisites](#), ensure you meet the following requirements for User Agent string implementation:

- Product listing on AWS Marketplace. See the [AWS Marketplace Seller Guide](#) for general information. For product-specific listing guide instructions:

Product Type	Listing Guide
Amazon Machine Image (AMI)	AMI Product Listing Guide
SaaS	SaaS Product Listing Guide
ML	ML Product Listing Guide
Professional Services	Professional Services Listing Guide

- Product that uses one or more of the [supported AWS services for User Agent string](#)
- Direct regular AWS API/CLI access from your application

Manual Implementation

Follow these steps to implement User Agent strings in your regular AWS API/CLI calls for Partner Revenue Measurement:

1. [Retrieve your product code from AWS Marketplace Management Portal](#).
2. Create the User Agent string using the format `APN_1.1/pc_<YOUR-PRODUCT-CODE>$`. Replace `<YOUR-PRODUCT-CODE>` with your actual alphanumeric product code. The `$` symbol marks the end delimiter.
3. Update your AWS SDK configuration to include the User Agent string. For complete implementation examples across different programming languages, see [User Agent Implementation Samples](#). For automated methods, see [Automated User Agent](#).

Basic example for AWS CLI:

```
# Set user-agent string for AWS CLI
export AWS_SDK_UA_APP_ID="APN_1.1/pc_5ugbbirmu7ud3u5hsipfzug61p$"

# Example EC2 API call with user-agent
aws ec2 run-instances --image-id ami-0abcdef1234567890 --instance-type t2.micro --
region us-east-1
```

Note

Replace 5ugbbrmu7ud3u5hsipfzug61p with your actual AWS Marketplace product code.

4. Test your implementation using AWS CloudTrail to verify the User Agent string is included:

```
# Check CloudTrail logs for user-agent string
aws logs filter-log-events \
  --log-group-name CloudTrail/YourLogGroup \
  --filter-pattern "APN_1.1" \
  --start-time 1640995200000
```

Look for the `userAgent` field in CloudTrail events to confirm your User Agent string is being sent with API calls.

5. Deploy to production and contact your AWS partner management team or [APN Support](#) (Partner Central login required) with your deployment details for validation.

Example CloudTrail event showing User Agent string:

```
{
  "eventTime": "2025-01-14T21:21:54Z",
  "eventName": "RunInstances",
  "eventSource": "ec2.amazonaws.com",
  "userAgent": "APN_1.1/pc_5ugbbrmu7ud3u5hsipfzug61p$ aws-cli/2.0.0",
  "sourceIPAddress": "203.0.113.12",
  "resources": [
    {
      "accountId": "123456789123",
      "type": "AWS::EC2::Instance"
    }
  ]
}
```

Note

User Agent strings must be implemented consistently across all regular AWS API/CLI calls to ensure accurate revenue attribution. For multi-region deployments, ensure the User

Agent string is included in regular AWS API/CLI calls across all regions where your solution operates.

 Note

Partner Revenue Measurement is intended to measure production workloads. Dev/test/staging environments can be used for validating your implementation before rolling out to production.

 Important

User Agent based attribution is evaluated monthly. Your partner solution must make at least one regular AWS API/CLI call per resource per month for continued attribution. If no calls are made on a resource in a given month, that resource does not contribute to revenue attribution for that month.

In scenarios where your partner solution does not make frequent regular AWS API/CLI calls, you can make non-mutating, read-only calls (such as `Describe*` operations) to demonstrate continued interaction between your partner solution and the AWS resource. Refer to the [included services](#) for supported API actions.

Infrastructure as Code Limitations

AWS CloudFormation / AWS CDK: CloudFormation does not support custom User Agent strings natively. When CloudFormation creates resources, it makes regular AWS API/CLI calls using its own service principal, and you cannot control the User Agent string in those calls. Use [Resource Tagging](#) for CloudFormation-deployed resources instead. If your CloudFormation template includes Custom Resources (Lambda-backed), the Lambda function code can include a User Agent string via the SDK approach.

Terraform: Terraform supports User Agent attribution via `provider_meta` with the `user_agent` argument (AWS provider `>= 6.27.0` or AWSCC provider `>= 1.67.0`). This is scoped to the declaring module only, ensuring correct attribution without collision across multiple partner modules. See [Terraform Implementation Sample](#) for full details.

Implementation Samples

This section provides code samples for implementing User Agent strings across different programming languages and AWS SDKs. Each sample demonstrates how to configure the User Agent string format `APN_1.1/pc_<YOUR-PRODUCT-CODE>$` for AWS services.

Note

Replace the example product code `5ugbbrmu7ud3u5hsipfzug61p` with your actual AWS Marketplace product code retrieved from the AWS Marketplace Management Portal.

Python (boto3)

```
import boto3
from botocore.config import Config

UA_STRING = "APN_1.1/pc_5ugbbrmu7ud3u5hsipfzug61p$"

# Create a config object with the custom user agent
session_config = Config(user_agent=UA_STRING)

# EC2 client
ec2 = boto3.client('ec2', config=session_config)

# S3 client
s3 = boto3.client('s3', config=session_config)
```

Java (AWS SDK v2)

```
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.client.config.SdkAdvancedClientOption;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.s3.S3Client;

public class AwsCustomUserAgentExample {
    private static final String UA_STRING = "APN_1.1/pc_5ugbbrmu7ud3u5hsipfzug61p$";
    private static final Region REGION = Region.US_WEST_2;

    public static void main(String[] args) {
```

```

        ClientOverrideConfiguration overrideConfig =
            ClientOverrideConfiguration.builder()
                .putAdvancedOption(SdkAdvancedClientOption.USER_AGENT_PREFIX, UA_STRING)
                .build();

        Ec2Client ec2 = Ec2Client.builder()
            .region(REGION)
            .overrideConfiguration(overrideConfig)
            .build();

        S3Client s3 = S3Client.builder()
            .region(REGION)
            .overrideConfiguration(overrideConfig)
            .build();
    }
}

```

.NET (AWS SDK)

```

using Amazon;
using Amazon.S3;
using Amazon.Runtime;

class Program
{
    static void Main(string[] args)
    {
        AWSConfigs.UseAlternateUserAgentHeader = true;
        string UA_STRING = "APN_1.1/pc_5ugbbrrmu7ud3u5hsipfzug61p$";

        var s3Config = new AmazonS3Config
        {
            RegionEndpoint = RegionEndpoint.USEast2,
        };
        var s3Client = new AmazonS3Client(s3Config);

        s3Client.BeforeRequestEvent += delegate (object sender, RequestEventArgs e)
        {
            if (e is WebServiceRequestEventArgs args)
            {
                args.Headers["User-Agent"] = UA_STRING;
            }
        };
    }
}

```

```

    }
  }
}

```

Node.js (AWS SDK v3)

```

const { EC2Client } = require("@aws-sdk/client-ec2");
const { S3Client } = require("@aws-sdk/client-s3");

const PRODUCT_CODE = "5ugbbirmu7ud3u5hsipfzug61p";
const REGION = "us-west-2";

function addPRMUserAgent(client, productCode) {
  client.middlewareStack.add(
    (next) => async (args) => {
      if (!args.request.headers) args.request.headers = {};
      const existing = args.request.headers["User-Agent"] || "";
      const prmUAStr = `APN_1.1/pc_${productCode}`;
      args.request.headers["User-Agent"] = existing
        ? `${existing} ${prmUAStr}`
        : prmUAStr;
      return next(args);
    },
    { step: "finalizeRequest", name: "prmUserAgent" }
  );
  return client;
}

const ec2Client = addPRMUserAgent(new EC2Client({ region: REGION }), PRODUCT_CODE);
const s3Client = addPRMUserAgent(new S3Client({ region: REGION }), PRODUCT_CODE);

```

Go (AWS SDK v2)

```

package main

import (
    "context"
    "net/http"

    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/s3"
)

```

```

const UA_STRING = "APN_1.1/pc_5ugbbirmu7ud3u5hsipfzug61p$"

type PRMUserAgentTransport struct {
    Transport http.RoundTripper
}

func (t *PRMUserAgentTransport) RoundTrip(req *http.Request) (*http.Response, error) {
    req.Header.Set("User-Agent", UA_STRING)
    return t.Transport.RoundTrip(req)
}

func main() {
    cfg, _ := config.LoadDefaultConfig(context.TODO(),
        config.WithRegion("us-east-2"),
        config.WithHTTPClient(&http.Client{
            Transport: &PRMUserAgentTransport{
                Transport: http.DefaultTransport,
            },
        }),
    )
    s3Client := s3.NewFromConfig(cfg)
    _ = s3Client
}

```

Ruby (AWS SDK v3)

```

require 'aws-sdk-ec2'
require 'aws-sdk-s3'

UA_STRING = 'APN_1.1/pc_5ugbbirmu7ud3u5hsipfzug61p$'
REGION = 'us-west-2'

Aws.config.update({
    region: REGION,
    user_agent_suffix: UA_STRING
})

ec2_client = Aws::EC2::Client.new
s3_client = Aws::S3::Client.new

```

Terraform

The Terraform AWS provider supports three ways to inject custom User Agent information:

Method	Scope	Recommended
provider_meta user_agent argument	Declaring module only	Yes
user_agent provider argument	Provider block	No
TF_APPEND_USER_AGENT env var	Global (all API calls)	No

provider_meta is scoped to the declaring module only, ensuring correct attribution without collision when multiple partner modules are used in the same Terraform configuration.

Prerequisites: Terraform ≥ 1.0 and AWS provider $\geq 6.27.0$ (or AWSCC provider $\geq 1.67.0$).

In your module's terraform block, add the provider_meta "aws" block with the PRM User Agent string:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 6.27.0"
    }
  }

  provider_meta "aws" {
    user_agent = [
      "APN_1.1/pc_5ugbbirmu7ud3u5hsipfzug61p$",
    ]
  }
}
```

Replace 5ugbbirmu7ud3u5hsipfzug61p with your actual product code. The \$ is a required end delimiter, do not omit it.

Important

User Agent attribution requires ongoing API or CLI interaction with AWS resources. Terraform typically interacts with resources only during `plan`, `apply`, and `destroy` operations. If the partner solution provisions static, long-running resources with limited ongoing API activity, consider using [tag-based attribution with Terraform](#) instead.

Note

Do not declare a `provider "aws" {}` block in your module. The provider configuration should be controlled by the root module (the customer). Your module should only use `provider_meta`.

`provider_meta` user-agent does **not** inherit to child modules. If your module calls other modules that also need attribution, each module must declare its own `provider_meta`. If both provider-level `user_agent` and `provider_meta` are present, the provider-level User Agent appears first in the header, followed by `provider_meta`.

Automated User Agent

For partner solutions that make multiple regular AWS API/CLI calls, you can automate User Agent string inclusion using AWS SDK Application ID settings instead of instrumenting every call individually.

Warning

Only configure User Agent strings for regular AWS API/CLI calls that are directly made by your partner solution. Do not configure User Agent strings for customer-initiated calls that are independent of your solution.

Note

If an Application ID is already configured, you can append your Partner Revenue Measurement User Agent string using a space as a delimiter. For example:

```
EXISTING_APP_ID APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$
```

Shared AWS Config File

Configure the User Agent string in the AWS shared configuration file (~/.aws/config):

Note

Replace 5ugbb1rmu7ud3u5hsipfzug61p with your product code. See [Product Code Retrieval](#). The \$ is the required end delimiter.

```
[default]
sdk_ua_app_id=APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$
```

Environment Variable

Set the User Agent string using environment variables for automated deployments:

Note

Replace 5ugbb1rmu7ud3u5hsipfzug61p with your product code. The \$ is the required end delimiter.

Linux/macOS:

```
export AWS_SDK_UA_APP_ID=APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$
```

Windows:

```
setx AWS_SDK_UA_APP_ID APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$
```

Python:

```
import os
os.environ['AWS_SDK_UA_APP_ID'] = 'APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$'
```

Node.js:

```
process.env.AWS_SDK_UA_APP_ID = 'APN_1.1/pc_5ugbb1rmu7ud3u5hsipfzug61p$';
```

JVM System Property (Java/Kotlin)

For Java and Kotlin applications, configure the User Agent string using JVM system properties:

Note

Replace 5ugbb1mu7ud3u5hsipfzug61p with your product code. The \$ is the required end delimiter.

```
java -Dsdk.ua.appId="APN_1.1/pc_5ugbb1mu7ud3u5hsipfzug61p$" -jar your-application.jar
```

Or set it programmatically:

```
System.setProperty("sdk.ua.appId", "APN_1.1/pc_5ugbb1mu7ud3u5hsipfzug61p$");
```

Support by AWS SDKs and Tools

The Application ID configuration methods described above are supported by most AWS SDKs and tools. For a complete list, see [Application ID support by AWS SDKs and tools](#). JVM system property settings are only supported by the AWS SDK for Java and the AWS SDK for Kotlin.

Validation

After configuring automated User Agent settings, verify the configuration:

1. Make a test AWS API call from your application
2. Check AWS CloudTrail logs to verify the User Agent string appears in the userAgent field
3. Confirm the format matches exactly: APN_1.1/pc_<YOUR-PRODUCT-CODE>\$

Included AWS Services

The following AWS services are supported for User Agent string implementation. User Agent strings must be included in all management or control-plane AWS API/CLI calls (i.e., calls that create, describe, modify, or delete AWS resources) to ensure proper revenue attribution.

For detailed information including specific API actions and regions supported for each service, download the [User Agent Included Services and API Actions \(CSV\)](#).

Note

Monthly API operations on resources are required for attribution to occur.

Note

Only management or control plane operations are supported. Data plane operations using AWS API/CLI are NOT supported.

Note

Partner Revenue Measurement intends to support all AWS services. We recommend that you instrument PRM on all AWS services and resources that your partner solution interacts with to avoid ongoing operational changes as service coverage expands. At this time, revenue attribution data is surfaced for the services listed below. Any partial spend captured on services not listed below is aggregated as "Misc".

User Agent String Included Services

Service Name	Product Service Code	Notes
Amazon API Gateway	AmazonApiGateway	None
Amazon AppStream	AmazonAppStream	Excludes User Fees
Amazon Athena	AmazonAthena	None
Amazon Bedrock	AmazonBedrock	Includes only Amazon models and Open Source models available in Amazon Bedrock
Amazon Bedrock AgentCore	AmazonBedrockAgentCore	Includes AgentCore RunTime, Browser Tool, Code Interpreter, Gateway, Identity, and Memory

Service Name	Product Service Code	Notes
Amazon Bedrock Service	AmazonBedrockService	None
Amazon Chime	AmazonChime	None
Amazon Chime	AmazonChimeFeatures	None
Amazon Chime	AmazonChimeServices	None
Amazon Chime	AmazonChimeVoiceConnector	None
Amazon CloudFront	AmazonCloudFront	Excludes data transfer costs and Lambda@Edge
Amazon CloudSearch	AmazonCloudSearch	None
Amazon CloudWatch	AmazonCloudWatch	Alarms and Logs
Amazon CodeWhisperer	AmazonCodeWhisperer	None
Amazon Cognito	AmazonCognito	Excludes Amazon Cognito add-ons
Amazon Connect	AmazonConnect	Includes Full Connect Unlimited AI and A La Carte; excludes Cases, Entity Resolution, Legacy Pinpoint Engagement, Meetings SDK, ContactLens, Chat, Email, Lex, Q in Connect, Tasks, Voice, Customer Profiles, Outbound Campaigns Processing, Telephony

Service Name	Product Service Code	Notes
Amazon Connect	ContactCenterTelecomm	None
Amazon Connect	ContactCenterTelecommKR	None
Amazon Connect	ContactCenterTelecommZA	None
Amazon Connect Customer Profiles	CustomerProfiles	None
Amazon Detective	AmazonDetective	None
Amazon DocumentDB (with MongoDB compatibility)	AmazonDocDB	None
Amazon DynamoDB	AmazonDynamoDB	None
Amazon DynamoDB Accelerator (DAX)	AmazonDAX	None
Amazon EC2	AmazonEC2	Includes Amazon EBS, EBS Snapshots, EC2 Mac, AWS Local Zones deployment, Savings Plan for ML instance types; excludes Capacity Block for ML
Amazon EC2	AmazonEC2OCPULicenseFees	None
Amazon ECR	AmazonECR	None
Amazon ECR Public	AmazonECRPublic	None
Amazon ECS	AmazonECS	Includes Fargate and AWS App Mesh

Service Name	Product Service Code	Notes
Amazon EKS	AmazonEKS	Excludes Fargate; includes AWS App Mesh
Amazon Elastic File System	AmazonEFS	None
Amazon ElastiCache	AmazonElastiCache	None
Amazon EMR	ElasticMapReduce	Includes AWS Local Zones deployment
Amazon FSx	AmazonFSx	None
Amazon GameLift Streams	AmazonGameLiftStreams	None
Amazon Grafana	AmazonGrafana	None
Amazon Interactive Video Service	AmazonIVS	None
Amazon Kendra	AmazonKendra	None
Amazon Keyspaces (for Apache Cassandra)	AmazonMCS	None
Amazon Kinesis Data Analytics	AmazonKinesisAnalytics	None
Amazon Kinesis Data Firehose	AmazonKinesisFirehose	None
Amazon Kinesis Data Streams	AmazonKinesis	Data Streams and Data Firehose
Amazon Kinesis Video Streams	AmazonKinesisVideo	None
Amazon Lex	AmazonLex	None

Service Name	Product Service Code	Notes
Amazon Managed Service for Prometheus	AmazonPrometheus	None
Amazon Managed Workflows for Apache Airflow (MWAA)	AmazonMWAA	None
Amazon MemoryDB for Redis	AmazonMemoryDB	Excludes Snapshot Storage
Amazon MQ	AmazonMQ	None
Amazon MSK	AmazonMSK	None
Amazon Neptune	AmazonNeptune	None
Amazon Omics	AmazonOmics	Excludes storage charges
Amazon OpenSearch Service	AmazonES	Includes Amazon Elasticsearch Service; excludes OpenSearch Serverless and OpenSearch Ingestion
Amazon Personalize	AmazonPersonalize	None
Amazon Q	AmazonQ	None
Amazon RDS OCPU License Fees	AmazonRDS OCPULicenseFees	None
Amazon Redshift	AmazonRedshift	Amazon Redshift Provisioned and Amazon Redshift Serverless
Amazon Relational Database Service (RDS)	AmazonRDS	Includes all RDS engines, Amazon RDS Custom, AWS Local Zones deployment; excludes Db2 licensing fees
Amazon Route 53	AmazonRoute53	Excludes Amazon Route 53 Resolver, Traffic Flow, and CIDR block storage

Service Name	Product Service Code	Notes
Amazon S3	AmazonS3	Includes storage cost only and all storage tiers; excludes Requests
Amazon S3 Glacier	AmazonGlacier	Excludes Glacier Deep Archive
Amazon Security Lake	AmazonSecurityLake	None
Amazon SES	AmazonSES	Excludes managed infrastructure hours
Amazon Simple Notification Service (SNS)	AmazonSNS	Excludes API request charges
Amazon Simple Queue Service (SQS)	AWSQueueService	Excludes API request charges
Amazon Timestream	AmazonTimestream	Excludes monitoring and analysis charges
Amazon Verified Permissions	AmazonVerifiedPermissions	None
Amazon WorkSpaces	AmazonWorkSpaces	Includes WorkSpaces Core; excludes third-party VDI management software license and third-party OS or software license on virtual desktop
Amazon WorkSpaces Secure Browser	AmazonWorkSpacesWeb	None
AWS Amplify	AWSAmplify	None
AWS App Fabric	AWSAppFabric	None
AWS App Runner	AWSAppRunner	None

Service Name	Product Service Code	Notes
AWS Application Migration Service	AWSApplicationMigrationSvc	None
AWS AppSync	AWSAppSync	None
AWS B2B Data Interchange	AWSB2Bi	None
AWS Backup	AWSBackup	None
AWS Billing Conductor	AWSBillingConductor	None
AWS Certificate Manager	ACM	None
AWS Certificate Manager	AWSCertificateManager	Includes AWS Certificate Manager Private CA
AWS CloudFormation	AWSCloudFormation	None
AWS CodeArtifact	AWSCodeArtifact	Excludes API request charges
AWS CodeBuild	CodeBuild	None
AWS CodePipeline	AWSCodePipeline	None
AWS Compute Optimizer	AWSComputeOptimizer	None
AWS Database Migration Service	AWSDatabaseMigrationSvc	None
AWS DataSync	AWSDataSync	None
AWS DevOps Agent Service	DevOpsAgent	None
AWS Direct Connect	AWSDirectConnect	Excludes AWS Local Zones
AWS Directory Service	AWSDirectoryService	None

Service Name	Product Service Code	Notes
AWS Elastic Disaster Recovery (DRS)	AWSElasticDisaster Recovery	None
AWS Elemental Inference	AWSElementalInference	None
AWS Elemental MediaConvert	AWSElementalMediaConvert	None
AWS Elemental MediaPackage	AWSElementalMediaPackage	None
AWS Elemental MediaStore	AWSElementalMediaStore	None
AWS Elemental MediaTailor	AWSElementalMediaTailor	None
AWS End User Messaging	AmazonPinpoint	Excludes Carrier Fees, Carrier Fee Count, Number Information (Carrier Lookup), Numbers Validated, Phone numbers, SMS Inbound Price
AWS Firewall Manager	AWSFMS	None
AWS Global Accelerator	AWSGlobalAccelerator	None
AWS Glue	AWSGlue	None
AWS HealthLake	AmazonHealthLake	Excludes FHIR data export and transformation
AWS Interconnect	AWSInterconnect	None
AWS IoT Core	AWSIoT	Excludes Registry operations

Service Name	Product Service Code	Notes
AWS IoT Device Management	IoTDevice Management	None
AWS IoT Greengrass	AWSGreengrass	None
AWS Key Management Service	awskms	Excludes cross-account request costs
AWS Lambda	AWSLambda	None
AWS Network Firewall	AWSNetworkFirewall	None
AWS Outposts	AWSOutposts	None
AWS Payment Cryptography	PaymentCryptography	Excludes following APIs under Requests usage: ListAliases, ListKeys, GetResourcePolicy, GetParametersForImport, GetParametersForExport, DeleteResourcePolicy, DeleteAlias, CreateKey, CreateAlias
AWS R53 App Recovery Controller	AWSR53App RecoveryController	None
AWS Resilience Hub	AWSResilienceHub	None
AWS Secrets Manager	AWSSecretsManager	Excludes API request charges
AWS Security Agent	SecAgent	None
AWS Step Functions	AmazonStates	None
AWS Storage Gateway	AWSSStorageGateway	None
AWS Transfer Family	AWSTransfer	Excludes AWSDataTransfer

Service Name	Product Service Code	Notes
AWS Transit Gateway	AmazonVPC	Includes Transit Gateway VPN, VPC, Peering, DirectConnect, DX; excludes data transfer costs
AWS WAF	awswaf	None
AWS Wisdom	AWSWisdom	None
Elastic Load Balancing	AWSELB	None

Revenue Attribution ID

A revenue attribution ID is a unique identifier that associates your AWS Marketplace offers and/or APN Customer Engagements (ACE) opportunities to a Marketplace product listing allowing AWS to determine the AWS revenue generated from those offers and opportunities. Revenue attribution builds on top of product-level Partner Revenue Measurement (PRM) implementation. PRM is your foundation — capturing total attributed revenue across your products or service delivery engagements. Revenue attribution is your deal-level overlay — assisting AWS in mapping revenue to specific offers and opportunities for which you are requesting AWS deal-level incentives.

After a revenue attribution ID is created, one or more AWS Marketplace offers or ACE opportunities can be associated with it. For each associated deal, you can specify the cost allocation percentage on a month-by-month basis. If you associate multiple offers or opportunities to one revenue attribution ID, AWS splits revenue proportionally based on the cost allocation percentages you specify.

Key Concepts

Revenue Attribution ID: A unique identifier created within AWS Partner Central that associates AWS revenue to one or more Marketplace offers and/or ACE opportunities. You can use a revenue attribution ID in two ways:

- **Deal-level overlay:** on your existing product-level PRM implementation — create a revenue attribution ID, associate your applicable Marketplace offers and/or ACE opportunities and specify deal-level cost allocations. AWS maps your already-measured product revenue to those specific deals based on your inputs. No changes to your existing resource tags or user agent strings are required. Note that deal level revenue attribution remains internal to AWS.
- **Standalone identifier:** used directly as a resource tag value or in a user agent string to associate AWS consumption to specific deals.

Association: The relationship between a Revenue Attribution ID and a Marketplace offer or ACE opportunity. Each association is configured by adding one or more monthly cost allocation entries — one per billing month for which the deal applies.

Cost Allocation Percentage: The portion of your product's total AWS consumption that is attributable to a specific deal for a specific billing month. Note that this input is required for

multi-tenant SaaS products, including partner-hosted components for hybrid deployments, where customers share infrastructure within partner's accounts.

Billing Month: The calendar month for which the cost allocation percentage entry applies. AWS attributes consumption attributed to your Revenue Attribution ID for that month according to the cost allocation percentages you specify.

Important

No implementation rework is required for existing PRM-enabled products. If you already implemented PRM using an AWS Marketplace listing's product code as the tag value or in the user agent string, you do not need to change that implementation to use a Revenue Attribution ID.

You can leverage this additional capability by creating a Revenue Attribution ID, associating it to the PRM-enabled product in Step 1 of the AWS Partner Central Revenue Attribution ID creation wizard, and providing your monthly deal-level cost allocation percentages in Step 2. Revenue Attribution IDs are designed to map product-level revenue to specific AWS Marketplace offers and/or ACE opportunities — eliminating rework.

Prerequisites

To create and use Revenue Attribution IDs, you must have:

1. Migrated to AWS Partner Central in the AWS Console
2. At least one AWS Marketplace product listing
3. A valid buyer account ID (for AWS Marketplace offer associations)
4. An opportunity in Launched stage with a customer AWS Account ID specified (for ACE opportunity associations)
5. Implemented at least one Partner Revenue Measurement capability — Resource Tagging or User Agent String — using either your AWS Marketplace product listing's product code (product-level PRM) or a Revenue Attribution ID as the unique identifier.

Partners with additional AWS Marketplace Seller Accounts that are connected via subsidiary account connections can create Revenue Attribution IDs for products that span multiple accounts.

Create and Edit Revenue Attribution IDs

This section covers how to create a Revenue Attribution ID, add and edit monthly cost allocation percentages, and implement PRM using the Revenue Attribution ID as the unique identifier.

Creating a Revenue Attribution ID

The Revenue Attribution ID creation flow guides you through three steps. To get started, navigate to the [Revenue attribution](#) under Sell menu in AWS Partner Central Console.

Step 1: Select your AWS Marketplace Product

On this page, select the AWS Marketplace product you want to associate with the Revenue Attribution ID. Only your AWS Marketplace products are eligible for selection. The product code from the selected listing is automatically associated with the Revenue Attribution ID.

If you want to associate a product owned by a subsidiary AWS account that is not directly available in the dropdown, you can manually enter the Marketplace product ID within the wizard.

Alternatively, if multiple AWS Marketplace products share the same underlying AWS resources — for example, a single deployment that spans more than one AWS Marketplace product listing — you can proceed without selecting a specific product. In this case, attribution applies across all consumption measured under the Revenue Attribution ID, regardless of which product you purchased.

Step 2: Associate offers or opportunities

On this page, you associate your Revenue Attribution ID with the AWS Marketplace offers and/or ACE opportunities and provide cost allocation percentages on a month-by-month basis.

Each entry includes the following information:

- **Offer ID or Opportunity ID:** The unique identifier of the deal you are associating. If an AWS Marketplace offer is already linked to an ACE opportunity, you only need to provide the Offer ID — AWS automatically attributes revenue to the associated opportunity.
- **Name:** A descriptive name for the associated entry to help you identify it in your dashboard.
- **Customer AWS Account ID:** The AWS account where the customer consumes your product. Required for multi-tenant SaaS products.
- **Billing Month:** The calendar month (for example, April 2026) for which the cost allocation percentage applies.

- **Cost Allocation %:** The portion of your product's total AWS consumption attributable to this deal for the specified billing month. Required for multi-tenant SaaS deployments and for any case where two or more deal associations are concurrently active in the same billing month.

To cover a deal that spans multiple billing months, add one entry per month. Each (Offer ID or Opportunity ID, Billing Month) combination is a separate entry. This per-month design allows you to adjust cost allocation percentages as deal economics change and ensures the prior month's allocation matches actual measured consumption.

The total Cost Allocation % across all entries for a given Revenue Attribution ID and the same Billing Month must not exceed 100%.

Step 3: Review and Submit

Review your Revenue Attribution ID name, associated Marketplace product, offers and opportunities, and the specified monthly cost allocation percentages. Submit to create the Revenue Attribution ID.

Adding and editing monthly cost allocation percentages

Cost allocation entries are managed per billing month. You can:

- **Add a new monthly entry** for a current or future billing month at any time.
- **Update a monthly entry** for the current or a future billing month at any time.
- **Update last month's entry** until the 7th of the current month. This window gives you time to review actual usage in AWS Cost Explorer before finalizing the prior month's allocation.

After the 7th of the current month, attribution for the prior billing month can no longer be modified. Updates to a current or future month's entry apply from the next monthly billing cycle. Historical attribution for prior months is not retroactively recalculated.

Implementing PRM with a Revenue Attribution ID

Important

No implementation rework is required for existing PRM-enabled products. If you already implemented PRM using an AWS Marketplace listing's product code as the tag value or in

the user agent string, you do not need to change that implementation to use a Revenue Attribution ID.

You can leverage this additional capability by creating a Revenue Attribution ID, associating it to the PRM-enabled product in Step 1 of the creation wizard, and providing your monthly deal-level cost allocation percentages in Step 2. Revenue Attribution IDs map product-level revenue to specific AWS Marketplace Offers and/or ACE opportunities — eliminating rework.

PRM implementation requires a unique identifier — either your AWS Marketplace product listing's product code or a Revenue Attribution ID — used as the value in a resource tag or user agent string. The two scenarios are:

Scenario 1: Product-level implementation (AWS Marketplace listing product code as the unique identifier)

For product-level implementation details, refer to the [Resource Tagging](#) or [User Agent String](#) documentation.

Scenario 2: Deal-level implementation (Revenue Attribution ID as the unique identifier)

If you implement PRM using a Revenue Attribution ID directly as the tag value or in the user agent string, AWS measures consumption at the deal level from the start. Use this approach for new implementations or when you want deal-level measurement without first establishing product-level PRM.

The Revenue Attribution ID format is `ra-<13 character string>` (for example, `ra-aabbccdde001`).

Resource Tagging

Add the tag to your resources:

- Tag Key: `aws-apn-id`
- Tag Value: `<RA ID>`
- Example: `ra-aabbccdde001`

See [Resource Tagging](#) for implementation details.

User Agent String

Configure the User Agent string format `APN_1.1/pc_<RA ID>$` for AWS services.

- Example: `APN_1.1/pc_ra-aabbccdde001$`

See [User Agent String](#) for implementation details.

Note

Only use a Revenue Attribution ID in your user agent string if you have not already implemented product-level PRM via user agent string. Multiple user agent identifiers are not supported. If you have already implemented product-level PRM via user agent string, use the deal-level overlay approach (Scenario 1) instead — no changes to your existing user agent string are required.

API Access

In addition to managing Revenue Attribution IDs through AWS Partner Central, you can manage them programmatically through the Partner Revenue Measurement APIs and the Marketplace Revenue Share APIs.

Partner Revenue Measurement APIs

Use the [Partner Revenue Measurement APIs](#) to create, retrieve, list, and update Revenue Attribution IDs and their monthly cost allocation percentages for associated AWS Marketplace offers and/or ACE opportunities. The API surface includes the following operations:

Revenue Attribution

Operation	Description
CreateRevenueAttribution	Creates a new Revenue Attribution ID.
GetRevenueAttribution	Retrieves the details of a Revenue Attribution ID by identifier.

Operation	Description
ListRevenueAttributions	Returns a paginated list of Revenue Attribution IDs.
UpdateRevenueAttribution	Updates the description of an existing Revenue Attribution ID.

Revenue Attribution Allocations

Operation	Description
StartRevenueAttributionAllocationsTask	Submits a batch of up to 250 monthly cost allocation changes for asynchronous processing.
GetRevenueAttributionAllocationsTask	Retrieves the status of an allocations task and per-record results.
ListRevenueAttributionAllocations	Returns a paginated list of monthly cost allocation entries for a given Revenue Attribution ID.
GetRevenueAttributionAllocation	Retrieves a single monthly cost allocation entry.

Marketplace Revenue Share APIs

Use the [Marketplace Revenue Share APIs](#) to specify what portion of their product sales are generated by AWS Marketplace for a specific duration (month, quarter, or year).

Marketplace Revenue Percentage

Operation	Description
CreateMarketplaceRevenueShare	Creates a Marketplace Revenue percentage resource for a specific AWS Marketplace product.
GetMarketplaceRevenueShare	Retrieves a Marketplace Revenue percentage resource by Marketplace ProductId.
ListMarketplaceRevenueShares	Returns a paginated list of Marketplace Revenue percentage resources.

Marketplace Revenue Percentage Allocations

Operation	Description
CreateMarketplaceRevenueShareAllocation	Creates an allocation declaring the Marketplace percentage and effective date window.
GetMarketplaceRevenueShareAllocation	Retrieves a single Marketplace Revenue percentage allocation.
UpdateMarketplaceRevenueShareAllocation	Updates a future-dated allocation or shortens the expiration of a current one.
ListMarketplaceRevenueShareAllocations	Returns a paginated list of allocations for a given Marketplace Revenue percentage.

Standard AWS tagging operations (TagResource, UntagResource, ListTagsForResource) are supported on the Marketplace Revenue percentage resource. Allocations do not support tags.

Troubleshooting Revenue Attribution IDs

This section describes common issues you might encounter when creating or managing Revenue Attribution IDs, and provides steps to resolve them.

Revenue Attribution ID not created successfully

If you are unable to create a Revenue Attribution ID:

- Verify that you have migrated to AWS Partner Central in the AWS Console.
- Confirm that you have at least one AWS Marketplace product listing.
- Ensure that you are selecting a product from your own account or a connected subsidiary account.
- Check that all required fields are completed (Marketplace product, at least one offer or opportunity association, billing month, and cost allocation percentage).

Cost allocation percentage entry rejected

If your cost allocation percentage entry is rejected:

- Verify that the total cost allocation percentage across all entries for the same Revenue Attribution ID and billing month does not exceed 100%.
- Confirm that each entry specifies a billing month and either an Offer ID or Opportunity ID.
- Check that each (Offer ID or Opportunity ID, Billing Month) combination is unique. Duplicate monthly entries for the same association are not allowed.
- Ensure that cost allocation percentages are between 0% and 100%.

Unable to update a prior month's cost allocation

If you cannot update a cost allocation entry for a prior billing month:

- Updates to the prior billing month's cost allocation are only accepted until the 7th of the current month.
- After the 7th, attribution for the prior billing month can no longer be modified.
- Historical attribution for prior months is not retroactively recalculated.
- To correct an error after the deadline, contact your AWS partner management team or APN Support (AWS Partner Central login required).

Offer or opportunity association failing validation

If your offer or opportunity association fails:

- For AWS Marketplace Offer associations, verify that the offer has a valid buyer account ID.
- For ACE opportunity associations, confirm that the opportunity is in Launched stage with a customer AWS Account ID specified.
- Verify that the Offer ID or Opportunity ID exists in AWS systems and belongs to your partner account.
- If an AWS Marketplace Offer is already linked to an ACE opportunity, you only need to provide the Offer ID.

Common implementation errors

The following table describes common implementation errors for Revenue Attribution IDs.

Method	Issue	Cause	Solution
Revenue Attribution ID	Cost allocation rejected	Total exceeds 100% for billing month	Ensure all entries for the same Revenue Attribution ID and billing month sum to 100% or less.
Revenue Attribution ID	Prior month update rejected	Past the 7th of current month	Updates to the prior month's allocation are only accepted until the 7th.
Revenue Attribution ID	Association fails validation	Offer or opportunity doesn't meet requirements	Verify that the offer has a buyer account ID or the opportunity is in Launched stage with a customer AWS Account ID.

Automation

Partner Revenue Measurement supports automation for each implementation method. Choose the automation approach that matches your implementation method.

AWS Marketplace Metering

AWS Marketplace Metering is a fully automated, zero-touch experience for partners. When you list an AMI or ML product on AWS Marketplace and customers purchase and use it, revenue attribution occurs automatically through existing product metadata. No additional implementation is required from partners.

For more information, see [AWS Marketplace Metering](#).

Resource Tagging Automation

Automate resource tagging using infrastructure as code tools and bulk tagging utilities. For detailed instructions, see [Automated Tagging](#) in the Resource Tagging chapter.

Supported automation tools:

- AWS CloudFormation
- AWS Cloud Development Kit (AWS CDK)
- Terraform
- AWS Tag Editor
- AWS CLI

User Agent String Automation

Automate User Agent string inclusion across all regular AWS API/CLI calls using AWS SDK Application ID configuration. For detailed instructions, see [Automated User Agent](#) in the User Agent String chapter.

Supported automation methods:

- Shared AWS config file (~/.aws/config)

- Environment variable (AWS_SDK_UA_APP_ID)
- JVM system property (Java/Kotlin)

Included AWS services

Partner Revenue Measurement supports specific AWS services for revenue attribution measurement. The supported services vary by implementation method.

- [AWS Marketplace Metering included services](#)
- [Resource Tagging included services](#)
- [User Agent String included services](#)

Best practices

Some AWS services require service-specific implementation steps for Partner Revenue Measurement. This section provides best practices for these services.

Amazon Bedrock - Resource Tagging

Amazon Bedrock uses *application inference profiles* as the taggable resource for Partner Revenue Measurement. You must create an application inference profile, tag it with the `aws-apn-id` tag, and then use that profile for all model invocations.

Understanding inference profiles

An inference profile is an Amazon Bedrock resource that specifies a foundation model and its associated AWS Regions for model invocation. Inference profiles enable cost management and resource measurement through cost allocation tags.

Amazon Bedrock offers the following types of inference profiles:

- **Cross-region (system-defined) inference profiles** – Predefined profiles that include multiple Regions to which requests for a model can be routed.
- **Application inference profiles** – User-created profiles to measure costs and model usage. You can create a profile that routes requests to one Region or to multiple Regions.

Warning

System-defined inference profiles do not support tagging. While system-defined (cross-region) inference profiles enhance flexibility in model usage by routing requests across multiple Regions, they do not support attaching custom tags for measuring, managing, and controlling costs across workloads and tenants. Only **application inference profiles** support the `aws-apn-id` tag required for Partner Revenue Measurement attribution. You must create an application inference profile to use Resource Tagging with Amazon Bedrock.

Prerequisites

- Access to supported foundation models through the Amazon Bedrock console or API.

- Your IAM role must have access to inference profile API actions. If your role has the `AmazonBedrockFullAccess` managed policy attached, you can skip this step. Otherwise, create a policy with the following permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel*",
        "bedrock:CreateInferenceProfile"
      ],
      "Resource": [
        "arn:aws:bedrock:*::foundation-model/*",
        "arn:aws:bedrock:*::inference-profile/*",
        "arn:aws:bedrock:*::application-inference-profile/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "bedrock:GetInferenceProfile",
        "bedrock:ListInferenceProfiles",
        "bedrock>DeleteInferenceProfile",
        "bedrock:TagResource",
        "bedrock:UntagResource",
        "bedrock:ListTagsForResource"
      ],
      "Resource": [
        "arn:aws:bedrock:*::inference-profile/*",
        "arn:aws:bedrock:*::application-inference-profile/*"
      ]
    }
  ]
}
```

Tagging an application inference profile

Follow these steps to create and tag an application inference profile using the AWS CLI.

Note

The tag value must use the format `pc:product-code`, where *product-code* is your AWS Marketplace product code. To retrieve your product code, see [Product Code Retrieval](#). Do **NOT** use the Product ID or UUID formatted product ID.

1. Create an application inference profile.

```
aws bedrock create-inference-profile \
  --inference-profile-name "my-partner-inference-profile" \
  --model-source "copyFrom=arn:aws:bedrock:us-east-1:123456789012:inference-profile/
us.amazon.nova-pro-v1:0"
```

Replace the Region, account ID, and inference profile ID with your values. The model source uses a `copyFrom` parameter that references a system-defined inference profile.

The output returns the inference profile ARN and status:

```
{
  "inferenceProfileArn": "arn:aws:bedrock:us-east-1:123456789012:application-
inference-profile/k1c3lwu201em",
  "status": "ACTIVE"
}
```

2. Tag the inference profile with your partner identifier.

```
aws bedrock tag-resource \
  --resource-arn "arn:aws:bedrock:us-east-1:123456789012:application-inference-
profile/k1c3lwu201em" \
  --tags Key=aws-apn-id,Value=pc:5ugbb1mu7ud3u5hsipfzug61p
```

Use the ARN from the previous step. Replace *5ugbb1mu7ud3u5hsipfzug61p* with your AWS Marketplace product code. For details on the tag format, see [Resource Tagging](#).

3. Verify the tag.

```
aws bedrock list-tags-for-resource \
  --resource-arn "arn:aws:bedrock:us-east-1:123456789012:application-inference-
profile/k1c3lwu201em"
```

The output confirms the tag:

```
{
  "tags": [
    {
      "key": "aws-apn-id",
      "value": "pc:5ugbbxmu7ud3u5hsipfzug61p"
    }
  ]
}
```

Important

All subsequent model invocations must use the tagged application inference profile ARN to ensure proper Partner Revenue Measurement attribution.

Listing application inference profiles

To list all application inference profiles in your account:

```
aws bedrock list-inference-profiles --type-equals "APPLICATION"
```

To get details for a specific profile:

```
aws bedrock get-inference-profile \
  --inference-profile-identifier "inference-profile-id"
```

Amazon EKS - Resource Tagging

Amazon EKS clusters run containerized applications on a set of nodes. Customers pay for EKS cluster hours and the underlying AWS resources including nodes (EC2 instances), load balancers, and EBS volumes. To measure revenue with Partner Revenue Measurement, you must tag both the Kubernetes cluster and its underlying AWS resources.

Note

The tag value must use the format `pc:product-code`, where *product-code* is your AWS Marketplace product code. To retrieve your product code, see [Product Code Retrieval](#).

Tagging the Kubernetes cluster

You can add tags to new or existing Kubernetes clusters using the Amazon EKS console, `eksctl`, AWS CLI, AWS API, or infrastructure-as-code tools like Terraform.

- **New clusters** – Apply tags during cluster creation using the `tags` parameter on the `CreateCluster` API action.
- **Existing clusters** – Apply tags using the `TagResource` API, or the **Tags** tab in the Amazon EKS console.
- **Terraform** – Use the `tags` argument when creating an EKS cluster. For more information, see [Terraform EKS cluster resource](#).

```
aws eks tag-resource \  
  --resource-arn arn:aws:eks:region:account-id:cluster/cluster-name \  
  --tags aws-apn-id=pc:5ugbbirmu7ud3u5hsipfzug61p
```

Tagging nodes within a node group

Amazon EKS clusters can schedule pods on any combination of self-managed nodes and EKS managed nodes. In all cases, ensure nodes are tagged with `aws-apn-id` using the format `pc:product-code`.

- **Managed nodes** – Use a custom launch template with the `TagSpecification` parameter to specify tags to apply to nodes (EC2 instances) in the node group. For example:

```
"TagSpecifications": [  
  {  
    "ResourceType": "instance",  
    "Tags": [  
      {  
        "Key": "aws-apn-id",  
        "Value": "pc:5ugbbirmu7ud3u5hsipfzug61p"  
      }  
    ]  
  }  
]
```

```
}  
]  
}  
]
```

You can launch a managed node group with a custom launch template using the EKS API, AWS CLI, CloudFormation, or the EKS console. For more information, see [Launch template support](#).

- **Self-managed nodes** – Apply tags by creating a node group with `eksctl` using the `tags` parameter. Tags are applied to all EC2 instances created as part of the node group. You can also apply tags using the AWS Management Console. For more information, see [Tagging your Amazon EC2 resources](#).

Tagging load balancers

The AWS Load Balancer Controller manages Elastic Load Balancers for a Kubernetes cluster.

- **Application Load Balancer (ALB)** – The controller creates an ALB when you create a Kubernetes Ingress. To tag ALBs, add the following annotation to the Ingress:

```
alb.ingress.kubernetes.io/tags: aws-apn-id=pc:5ugbb1mu7ud3u5hsipfzug61p
```

For more information, see [Application load balancing on Amazon EKS](#).

- **Network Load Balancer (NLB)** – The controller creates an NLB when you create a Kubernetes Service of type `LoadBalancer` using IP targets. To tag NLBs, add the following annotation to the Service:

```
service.beta.kubernetes.io/aws-load-balancer-additional-resource-tags: aws-apn-id=pc:5ugbb1mu7ud3u5hsipfzug61p
```

For more information, see [Network load balancing on Amazon EKS](#).

Tagging EBS volumes

The Amazon EBS Container Storage Interface (CSI) driver provides a CSI interface that allows Amazon EKS clusters to manage the lifecycle of EBS volumes. To add tags to dynamically provisioned EBS volumes, use the `--extra-tags` command option in the CSI driver. For detailed instructions, see the [Amazon EBS CSI driver documentation](#).

Fargate on EKS

Fargate on EKS is not supported for Partner Revenue Measurement resource tagging.

Isolating partner solution resources on a shared EKS cluster

If a customer runs both partner solution workloads and other workloads in the same EKS cluster, you cannot tag the cluster itself (the control plane). Instead, tag only the load balancers and the nodes running partner solution workloads. To isolate the workloads, use Kubernetes affinities and taints.

1. Create a partner-solution-only node group.

- Tag this node group following the instructions in [Tagging nodes within a node group](#).
- Add a taint with effect NO_SCHEDULE.
- Add a label (for example, partner-solution: "true").

2. Create partner solution pods.

- Add a toleration to the pods with a key/value matching the taint from step 1 and an effect of NoSchedule.
- Add a requiredDuringSchedulingIgnoredDuringExecution affinity with a matchExpression that matches the label from step 1.

Steps 1 and 2a prevent non-partner-solution pods from deploying on partner solution nodes. Steps 1b and 2b prevent partner solution pods from deploying on non-partner-solution nodes. No changes are required to existing non-partner-solution pods or nodes.

For more information, see:

- [Taints and Tolerations](#)
- [Node Affinity](#)
- [Taints on EKS managed node groups](#)

Demo

Explore interactive click-through demos that walk you through AWS Partner Revenue Measurement implementation steps. These guided walkthroughs let you experience the setup process in a hands-on format without requiring access to a live AWS environment.

Access the demos here: [AWS Partner Revenue Measurement Click-Through Demos](#)

AWS Samples

This repository, maintained by AWS Partner Solution Architects (PSAs), contains scripts in the form of AWS CloudFormation templates for automated AWS resource tagging and tag remediation. These are intended to help you with AWS Partner Revenue Measurement governance.

You can use these scripts to build your own automation for your specific situation. Make sure to test in a sandbox environment and follow your organization's SDLC practices for production deployment.

These scripts are meant to ease the adoption of AWS Partner Revenue Measurement by providing ideas for how to operationalize the implementation of PRM tagging requirements.

Note

Under the Shared Responsibility Model, partners and customers using this project need to enhance the templates to adapt them to their own environments, including but not limited to securing IAM permissions, enabling encryption, implementing log immutability, and monitoring security events.

Access the samples repository: [AWS Partner Revenue Measurement Samples on GitHub](#)

FAQs

This section provides frequently asked questions about Partner Revenue Measurement, organized by audience. AWS Partners implementing Partner Revenue Measurement should start with the [Partner FAQ](#). AWS customers who use partner solutions in their accounts should see the [Customer FAQ](#).

Partner FAQ

The following questions are frequently asked by AWS Partners implementing Partner Revenue Measurement.

1. General Questions

1.1 What is Partner Revenue Measurement?

Partner Revenue Measurement is a set of capabilities that enables AWS Partners to measure the AWS service consumption driven by their solutions and quantify their impact on overall AWS revenue. These capabilities empower AWS Partners to better understand their AWS revenue impact and product consumption patterns. Partner Revenue Measurement offers three implementation options: [AWS Marketplace Metering](#), [Resource Tagging](#), and [User Agent String](#).

1.2 Which AWS services are supported by Partner Revenue Measurement?

Partner Revenue Measurement supports AWS services across implementation methods. The supported services vary by method. See [AWS Marketplace Metering included services](#), [Resource Tagging included services](#), and [User Agent String included services](#) for complete lists.

1.3 Where do I find my AWS Marketplace product code?

Log in to **AWS Marketplace Management Portal**, navigate to your **Products** page, select your product, and find the product code in the **Product Summary** section. The product code format is typically a long alphanumeric string like: **5ugbbrmu7ud3u5hsipfzug61p**. Please do **NOT** use the Product ID or the UUID formatted product ID from the AWS Marketplace listing. For further information, refer to [Retrieve your product code](#).

1.4 What architecture patterns does Partner Revenue Measurement support?

Partner Revenue Measurement supports three architecture patterns: 1) Partner Account - all components in partner's AWS account/VPC, 2) Customer Account - all components in customer's AWS account/VPC, 3) Hybrid - components distributed across both partner and customer accounts/VPCs.

1.5 How do I get support for Partner Revenue Measurement implementation?

Contact your AWS partner management team or [APN Support](#) (Partner Central login required) for validation assistance and support with your Partner Revenue Measurement implementation.

1.6 How should I handle if there is another tag on the AWS resource?

Since an AWS resource can only have one tag with the `aws-apn-id` key, only one partner identifier is allowed per resource. For multi-partner scenarios where multiple partners operate on the same AWS resource, consider using the [User Agent String](#) method instead. If you must use resource tagging, coordinate with the other partner and the customer to determine tag ownership before making changes.

1.7 Who can remove tags?

Any user that has access to the account can remove a tag. Both customers and partners (if they have access to the customer's account) can remove tags.

1.8 Which regions are currently supported?

Partner Revenue Measurement currently supports only Commercial regions, not European Sovereign Cloud (ESC) or US GovCloud / Amazon Dedicated Cloud (ADC).

1.9 How often does my partner solution need to make regular AWS API/CLI calls for User Agent based attribution?

Your partner solution must make at least one regular AWS API/CLI call per resource per month. Attribution is evaluated on a monthly billing cycle. If no calls are made on a resource in a given month, that resource does not contribute to revenue attribution for that month. Attribution resumes the next month a qualifying call is made. In scenarios where your partner solution does not make frequent calls, you can use non-mutating, read-only calls (such as `Describe*`

operations) to demonstrate continued interaction. Refer to the [included services](#) for supported API actions.

1.10 What are my options if the customer's environment does not permit additional resource tags?

Use the [User Agent String](#) method instead. User Agent strings do not require adding user-defined tags to any AWS resource, do not consume the customer's tag quota, and do not interfere with existing tag policies. The User Agent string is captured in AWS CloudTrail logs, which also provides the customer with operational visibility into partner solution activity for auditing and operational excellence purposes.

2. Troubleshooting

For troubleshooting guidance across all implementation methods, see [Troubleshooting Partner Revenue Measurement](#).

3. Additional FAQs

For additional FAQs, see the Partner Revenue Measurement FAQs on [AWS Partner Central](#) (login required).

Customer FAQ

1. What is Partner Revenue Measurement?

Partner Revenue Measurement is a set of AWS capabilities that enables AWS and its Partners to measure the AWS service consumption Partners drive through their solutions or services engagements across applicable accounts. It gives Partners and AWS a clear, data-driven view of Partner impact so AWS can better support Partners as they deliver value to our mutual customers.

2. Why is my Partner asking to implement Partner Revenue Measurement?

Partner Revenue Measurement allows AWS to recognize Partners who are driving customer success. To accurately measure the value these Partners are driving, Partner Revenue Measurement will attribute your AWS consumption to Partner engagement. This connection enables AWS to enhance the tools and programs that support our partners and our mutual customers.

3. What are the customer benefits of Partner Revenue Measurement?

With Partner Revenue Measurement, customers can map costs to specific Partner products and solutions, better manage their overall portfolio spending, and make more informed decisions about how to allocate resources. Additionally, Partner Revenue Measurement provides operational visibility through AWS CloudTrail logs, giving customers an audit trail of partner solution activity for compliance and operational efficiency.

4. What impact does Partner Revenue Measurement have on my AWS environment?

Partner Revenue Measurement has no impact on your infrastructure, security posture, and billing structure. Partner Revenue Measurement only enables AWS to recognize and measure the Partner's influence on that environment. Partner Revenue Measurement works by associating a Partner's contributions with existing AWS resource usage through lightweight mechanisms such as [resource tagging](#) or [user agent strings](#). There is no additional software to install, no performance overhead, and no changes to how workloads run. With this information, AWS can better support both our Partners and our mutual customers.

5. What specific data does AWS collect?

Partner Revenue Measurement uses a tag or identifier--a simple label associated with your Partner's product--to correlate existing billing and usage information that AWS already maintains as part of its standard service operations. A customer's Partner does not receive their raw billing data; AWS provides them only with aggregated, attributed revenue metrics across all customers so a Partner can understand how their product drives AWS service consumption. AWS will collect an AWS Marketplace product code and resource tag, AWS Resource Name (ARN), API operation, ARN usage spend, Service Type, API operation or tag creation date and time, and the AWS Account ID hosting the AWS resource.

6. If I allow my Partner to implement Partner Revenue Measurement via resource tagging, how does Partner Revenue Measurement impact my tagging strategy?

By allowing your Partner to implement Partner Revenue Measurement via resource tagging, a single tag is attributed to relevant resources. This consumes one of your fifty available user-defined tag slots. If you enforce tag policies through AWS Organizations, you will need a one-time update

to allow for the tag key. You remain in full control: The Partner can only apply tags using the IAM permissions you explicitly grant, and you can revoke that access at any time. If the additional tag or policy change does not fit your environment, ask your Partner to use an alternate Partner Revenue Measurement method such as the [user agent string](#), which requires no tags in your account. View [this guide](#) for more information on managing resource tags.

7. How long do resource tags stay in my environment?

You have complete control over how long tags remain in your environment. Tags remain on resources until either you (or an authorized user) remove them, or the resource itself is terminated. Any user with the appropriate permissions in your account can remove a tag at any time. You are always the decision-maker on which Partner tags can be used on your resources. When a tag is removed, revenue attribution for the Partner's solution stops.

8. Does Partner Revenue Measurement give my Partner or AWS new access to my account?

No, Partner Revenue Measurement does not grant AWS or a Partner access to a customer's accounts, resources, or application code.

9. Is my usage shared with my Partner? What information is shared?

The Partner Revenue Measurement data AWS collects is only shared with your Partner on an aggregated basis. AWS does not share any Partner Revenue Measurement data specific to an individual customer with a Partner.

10. In which AWS regions is Partner Revenue Measurement supported?

Today, Partner Revenue Measurement is supported only in [commercial regions](#). AWS is working to expand coverage to sovereign and government cloud regions in the future, and your Partner or AWS account team can keep you informed on timing as it becomes available. If you run workloads in both commercial and government regions, Partner Revenue Measurement can be enabled today for your commercial accounts while you wait for expanded coverage.

Troubleshooting Partner Revenue Measurement

This section provides troubleshooting guidance for each Partner Revenue Measurement implementation method:

- [AWS Marketplace Metering Issues](#)
- [Resource Tagging Issues](#)
- [User Agent String Issues](#)

AWS Marketplace Metering Issues

AWS Marketplace Metering is a fully automated, zero-touch experience. Revenue attribution occurs automatically when customers purchase and use your AMI or ML product through AWS Marketplace. No additional implementation is required from partners.

If you are experiencing issues with revenue attribution for your AWS Marketplace Metering products, open a support ticket through [APN Support](#) (Partner Central login required). Include the following details:

- AWS account ID
- AWS Marketplace product code
- Product type (AMI or ML)
- Description of the issue and expected behavior

Resource Tagging Issues

Tags not providing revenue attribution

If resource tags are not generating revenue attribution:

Verify tag implementation

1. Check tag key is exactly: **aws-apn-id** (lowercase)
2. Verify tag value format: **pc:product-code**

3. Confirm product code matches AWS Marketplace listing (see [Product Code Retrieval](#))
4. Ensure resources are in [supported services](#)
5. Check that resources are actively consuming AWS services and incurring spend. For example, IAM is a no-cost AWS service, so tagging IAM resources does not generate revenue attribution. Focus on tagging resources that incur charges such as EC2 instances, S3 buckets with storage, RDS databases, or Lambda functions with invocations
6. Verify tags are applied correctly using [AWS Tag Editor](#) or reach out to your AWS partner management team or [APN Support](#) (Partner Central login required) for assistance

Tag conflicts with other partners

Since an AWS resource can only have one tag with the `aws-apn-id` key, only one partner identifier is allowed per resource. If another partner's tag exists on a resource, resource tagging creates a conflict.

For multi-partner scenarios where multiple partners operate on the same AWS resource, consider using the [User Agent String](#) method instead. Each partner can independently use their own identifier within their respective regular AWS API/CLI calls without conflicts.

If you must use resource tagging, coordinate with the other partner and the customer to determine tag ownership before making changes.

User Agent String Issues

User Agent string not appearing in CloudTrail

If your User Agent string is not visible in CloudTrail logs:

Verify User Agent implementation

1. Confirm the format is exactly: `APN_1.1/pc_<YOUR-PRODUCT-CODE>$`
2. Verify the `$` end delimiter is present and not stripped by your shell or runtime
3. Confirm product code matches AWS Marketplace listing (see [Product Code Retrieval](#))
4. Ensure the User Agent string is applied to the correct AWS SDK client configuration, not just one service client

5. Check CloudTrail is enabled and logging the relevant API calls in the correct region
6. Verify the `userAgent` field in CloudTrail events contains your string

Common format errors

Common User Agent String Format Issues

Issue	Cause	Solution
Missing end delimiter	The <code>\$</code> character was omitted or escaped by the shell	Ensure the string ends with <code>\$</code> . Use single quotes or escape appropriately for your shell
Wrong prefix	Using incorrect prefix format	Use exactly <code>APN_1.1/pc_</code> as the prefix
Product code mismatch	Using Product ID or UUID instead of product code	Retrieve the alphanumeric product code from AWS Marketplace Management Portal (see Product Code Retrieval)
No API operations on resources	Resources not receiving API calls in a given month	Ensure your product performs at least one API operation on an AWS resource per month for attribution

CloudTrail Logs Verification

Use the following command to verify your User Agent string appears in CloudTrail logs:

```
aws logs filter-log-events \  
  --log-group-name CloudTrail/YourLogGroup \  
  --filter-pattern "APN_1.1" \  
  --start-time 1640995200000
```

Look for the `userAgent` field in CloudTrail events. The format should match: `APN_1.1/pc_<YOUR-PRODUCT-CODE>$`

Validation and Testing

AWS Partner Team validation

For official validation, contact your AWS partner management team or [APN Support](#) (Partner Central login required):

- Include: AWS account ID, region, product code, sample resource ARN
- Provide: Tag screenshots or CloudTrail log excerpts and test timestamps
- Allow: 3-5 business days for validation response

Common Implementation Errors

Common Partner Revenue Measurement Implementation Issues

Method	Issue	Cause	Solution
Resource Tagging	Tags not working	Wrong tag format	Use <code>aws-apn-id</code> key with <code>pc:product-code</code> value
Resource Tagging	No revenue attribution	Resources not incurring spend	Ensure resources are actively consuming AWS services and incurring charges
Resource Tagging	Product code mismatch	Incorrect product code	Verify code in AWS Marketplace Management Portal (see Product Code Retrieval)
User Agent String	String not in CloudTrail	SDK not configured correctly	Verify SDK client configuration includes User Agent string for all service clients
User Agent String	Missing delimiter	\$ stripped by shell	Use single quotes or escape the \$ character appropriately
Marketplace Metering	No attribution	Product not purchased via Marketplace	Ensure customers purchase and use the product through AWS Marketplace. Open

Method	Issue	Cause	Solution
			a support ticket via APN Support

Contact Information

For any further questions about Partner Revenue Measurement (PRM) compliance requirements, please contact your AWS Partner Development team or [AWS Partner Network support](#).

Document history for the Partner Revenue Measurement User Guide

The following table describes the documentation releases for Partner Revenue Measurement.

Change	Description	Date
Expanded User Agent included services	Expanded the User Agent included services list from 24 to 119 AWS services. Updated the downloadable API actions CSV with supported actions and Regions for each service. Clarified that User Agent strings apply to management or control-plane API calls and that data plane operations are not supported. Spend on services not listed is aggregated as Misc.	August 31, 2026
Restructured FAQs by audience; added Customer FAQ	Restructured the FAQs chapter into audience-specific pages: Partner FAQ and Customer FAQ. Added a new Customer FAQ covering the purpose of Partner Revenue Measurement, customer benefits, environment and billing impact, data collected, tagging strategy considerations, tag lifecycle, account access, data sharing, and supported regions.	August 19, 2026

Added Revenue Attribution ID service user guide	Added new Partner Revenue Attribution ID service user guide for PRM.	June 30, 2026
Added Demo and AWS Samples sections; updated Resource Tagging included services	Added Demo section with click-through walkthroughs and AWS Samples section with CloudFormation automation examples. Added Amazon DynamoDB Accelerator (DAX), Amazon Elastic VMware Service (Amazon EVS), Amazon WorkSpaces Core Managed Instances, and AWS End User Messaging to Resource Tagging included services. Updated Amazon Bedrock notes to clarify supported model types.	May 18, 2026
Added best practices chapter and additional FAQs; updated Terraform User Agent guidance	Added best practices chapter covering Bedrock and EKS integration. Added additional FAQs section.	April 13, 2026
Updated Terraform User Agent implementation guidance	Updated Terraform section to use <code>provider_meta</code> with the <code>user_agent</code> argument (AWS provider <code>>= 6.27.0</code>).	April 8, 2026

[Added User Agent String and AWS Marketplace Metering implementation methods; restructured documentation](#)

Added User Agent String and AWS Marketplace Metering as new implementation methods. Restructured documentation to organize content by implementation method: AWS Marketplace Metering, Resource Tagging, and User Agent String. Updated Automation and Included AWS Services as hub pages linking to method-specific content.

April 2, 2026

[Initial release of Partner Revenue Measurement with ResourceTagging capability](#)

Initial release of the PartnerRevenueMeasurement User Guide

January 29, 2026